

令和 8 年度

博士前期課程学位論文

**Simulation-Based Rash Behavior Prediction of Surrounding
Vehicles Using Multi-Modal Sensor Fusion**

九州工業大学

大学院情報工学府情報創成工学専攻

学籍番号 **246E0701**

氏名 **Sirpy PALANISWAMY**

指導教員 楊 波

副指導教員 林 英治, 大竹 博

研究室 楊波研究室

TABLE OF CONTENTS

Table of Contents	i
List of Figures and Tables	iii
Acknowledgment	v
Abstract of Thesis	vi
Chapter 1 Introduction.....	1
1.1. Background	1
1.2. Problem Statement	3
1.3. Research Objectives	5
1.4. Significance of Study	5
1.5. Organization of Dissertation	6
Chapter 2 Literature Review	7
2.1. Ego Centric Aggressive Driving Detection	7
2.2. Multi-Modal Sensor Fusion for Autonomous Vehicles	7
2.3. Trajectory and Behavior Prediction	8
2.4. Surrounding Vehicle Intent	9
2.5. Simulation Platforms for Autonomous Driving Research	10
2.6. Summary and Identified Research Gap	12
Chapter 3 Dataset Construction	13
3.1. Simulation Platform Selection.....	13
3.2. Software and Hardware Environment	15
3.3. Scene and Event Generation Protocol	16
3.4. Engineering Corrections to the Sensor Pipeline.....	18
3.5. Sensor Modalities and Synchronization	19
3.6. Label Schema	22
3.7. Dataset Statistics and Train/Validation/Test Split.....	22
3.8. Reproducibility	23
3.9. Summary	24
Chapter 4 Proposed System Architecture	25
4.1. Overview	25
4.2. Design Rationale: Why Feature-Level (Early) Fusion.....	25
4.3. Model Architecture.....	26

4.4. Training Configuration.....	28
4.5. Comparison Models	29
4.6. Summary	30
Chapter 5 Experiments and Results.....	31
5.1. Overview	31
5.2. Implementation Details	31
5.3. Evaluation Metrics	30
5.4. Model 1: Kinematic LSTM Baseline	32
5.5. Model 2: Early Fusion V1	34
5.6. Model 3: Early Fusion V2 (Proposed Model)	35
5.7. Ablation Study.....	38
5.8. Overfitting Analysis	39
5.9. Generalization to Unseen Road Layouts	40
5.10. Time-to-Collision Safety Analysis.....	41
5.11. Computational Efficiency.....	45
5.12. Discussion	46
5.13. Summary	46
Chapter 6 Conclusion and Future Work.....	48
6.1. Summary of Contributions	48
6.2. Limitations.....	48
6.3. Future Work	49
6.4. Concluding Remarks	50
REFERENCES.....	51

LIST OF FIGURES AND TABLES

FIGURES

- Page 3** *Fig 1.1 Hybrid Traffic Situation, constituting both AV and Human driven system on road*
- Page 11** *Fig 2.1 MetaDrive simulation platform [19]*
- Page 17** *Fig 3.1. Top-Down Sensor Suite Coverage*
- Page 20** *Fig 3.2. Surround camera images collected through the RGB camera modality.*
Fig 3.3. Surround camera images collected through the semantic camera modality.
Fig 3.4. BEV LiDAR Grid 3 channel grid representation
- Page 21** *Fig 3.5. Raw LiDAR Point Cloud Representation.*
- Page 23** *Fig 3.6. Dataset overview visualization*
- Page 24** *Fig 3.7. Feature Caching Statistics for the Dataset*
- Page 26** *Fig 4.1. Early Fusion V2 architecture diagram. Four modality-specific encoders (BEV LiDAR, RGB cameras, semantic cameras, kinematics) feed a 192-dimensional fused feature vector into a Bidirectional GRU and classification head. Trainable components (BEV CNN, projection heads, GRU, classification head) and frozen components (MobileNetV3-Small backbones) are color-coded separately; total model size is 492,193 parameters.*
- Page 33** *Fig 5.1. Training Curve and the F1 score of Kinematic LSTM Model*
- Page 34** *Fig 5.2. Confusion Matrix*
Fig 5.3. ROC Curve
- Page 35** *Fig 5.4. Training Curve and F1 Score of Early Version V1*
Fig 5.5. Confusion Matrix
Fig 5.6. ROC Curve
- Page 36** *Fig 5.7. Training Curve and F1 Score of Primary Early Version V2*
- Page 37** *Fig 5.8. Confusion Matrix*
Fig 5.9. ROC Curve
- Page 38** *Fig 5.10. The Proposed model's metric headline scores.*
- Page 39** *Fig 5.11. Ablation Comparison of the three Models – Kinematic LSTM, Early Fusion V1, Early Fusion V2 – respectively.*

- Page 43** *Fig 5.12. Violin plot of minimum TTC, grouped by prediction outcome (TP/FP/FN/TN) on the test set, with a dashed line marking the 3-second danger threshold.*
- Page 44** *Fig 5.13 Mean TTC at true-positive detections plotted against classification threshold from 0.05 to 0.95, with true-positive count overlaid as a bar series.*
- Page 45** *Figure 5.14: Histogram comparing detected (TP) and missed (FN) event frames by minimum TTC, with the 3-second safety-critical zone marked.*

TABLES

- Page 14** *Table 3.1. MetaDrive vs. CARLA comparison*
- Page 15** *Table 3.2. Software Stack*
- Page 21** *Table 3.3. Kinematic Feature Vector Composition*
- Page 37** *Table 5.1 Early Fusion V2 test-set classification report*
- Page 38** *Table 5.2: Model comparison summary*

ACKNOWLEDGMENT

As I would like to express my sincere gratitude to my supervisor, Associate Professor Bo Yang, for their invaluable guidance, encouragement, and constructive feedback throughout the course of this research. Their expertise and patience were instrumental in shaping both the direction and depth of this work.

I am grateful to the faculty and staff of the Graduate School of Computer Science and Systems Engineering, Kyushu Institute of Technology, for providing a stimulating research environment and the computational resources necessary to carry out the simulation-based experiments described in this dissertation.

I extend my thanks to my fellow laboratory members for the insightful discussions, technical advice, and camaraderie that made the research process both productive and enjoyable. Special thanks are also due to the reviewers and committee members whose critical feedback helped strengthen this work considerably.

I acknowledge the developers of the MetaDrive simulation platform and the open-source communities behind PyTorch, MobileNetV3, and the broader autonomous driving research ecosystem, whose tools and datasets made this research possible.

Finally, I owe a deep debt of gratitude to my family and friends for their unwavering support, patience, and encouragement throughout my graduate studies. This work would not have been possible without them.

ABSTRACT OF THESIS

Rash driving behavior — characterized by sudden, aggressive maneuvers such as abrupt lane changes, sharp braking, and unsafe overtaking — poses a significant safety risk to autonomous vehicles (AVs) operating in shared traffic environments. Existing approaches to aggressive driving detection are predominantly ego-centric, relying on the motion history of the vehicle itself. This dissertation addresses a fundamentally different and practically important problem: predicting whether a nearby third-party vehicle will execute a rash maneuver in the immediate future, using only the ego AV's onboard multi-modal sensor suite.

A simulation-based framework is developed using the MetaDrive driving simulator to construct a labelled dataset comprising four simultaneous sensor modalities — Bird's-Eye View (BEV) LiDAR occupancy grids, six-view RGB cameras, six-view semantic segmentation maps, and a 19-dimensional kinematic vector — captured at 10 Hz across 1,000 procedurally varied driving scenarios. Each frame carries a binary rash-event label derived from a scripted behavioral trigger, yielding a dataset of approximately 20,000 labelled samples with an 83.9%/16.1% class imbalance.

Three predictive models are designed and evaluated. The first, Kinematic LSTM, processes only the kinematic modality through a Bidirectional GRU and establishes a strong single-modality baseline (test F1 = 0.81). The second, Early Fusion V1, augments the kinematic branch with RGB camera features, providing modest improvement. The primary contribution, Early Fusion V2, implements a feature-level (early) fusion architecture in which four modality-specific encoders — a trainable BEV CNN, frozen MobileNetV3-Small backbones for RGB and semantic streams, and a small kinematic projection head — produce a 192-dimensional fused feature vector per frame. A sliding temporal window of ten frames is processed by a two-layer Bidirectional GRU, whose output is mapped to a rash-event probability through a

lightweight classification head. Class-weighted binary cross-entropy loss and cosine annealing with early stopping are employed to address class imbalance and overfitting.

Early Fusion V2 achieves a test F1 score of 0.90, AUC of 0.97, precision of 0.88, and recall of 0.92, representing a substantial improvement over the kinematic baseline and demonstrating that multi-modal spatial context — particularly the BEV LiDAR and semantic modalities — provides complementary predictive signal beyond kinematic motion alone. Time-to-Collision analysis further confirms that the model generates actionable alarms well in advance of the scripted rash event onset.

This work demonstrates the feasibility of simulation-based multi-modal rash driving prediction for third-party vehicles from an ego AV perspective, and provides a reusable dataset construction pipeline, a validated fusion architecture, and a set of evaluation protocols applicable to future research in proactive AV safety systems.

Chapter 1

Introduction

1.1 Background

Autonomous Vehicles (AV) in the current automotive industry face the issue of navigating traffic through various situations and awareness. In the current automotive industry, the level of autonomous vehicles has been split into 6 different levels by the *Society of Automotive Engineers (SAE J3016)*, ranging from partial automation to true autonomy [1]. The capability of each level is classified by SAE as follows:

(1) Level 0: No Driving Automation

The human driver performs the entire dynamic driving task (DDT), with few automated active safety features, including emergency braking.

(2) Level 1: Driver Assistance

The vehicle here provides continuous support for either lateral control (steering) or longitudinal control (braking/acceleration) but never both simultaneously.

(3) Level 2: Partial Driving Automation

The vehicle on-board system here provides simultaneous, continuous lateral and longitudinal control (e.g., lane-centering combined with adaptive cruise control). However, the human driver is still entirely responsible for monitoring the environment and supervising the system in case of any emergencies.

(4) Level 3: Conditional Driving Automation

The Automated Driving System (ADS) performs the entire DDT and actively monitors the driving environment within a specific Operational Design Domain (ODD). The human driver does not need to look at the road but needs to remain receptive to a “request to intervene” and take back control when the situation requires it.

(5) Level 4: High Driving Automation

The ADS performs the entire DDT and monitors the environment with zero human interaction or intervention, and in the case of emergency, the ODD ends and the system fallbacks to a minimal risk condition (e.g., pulling over to the safety lines). Operation is still bound by specific geographic regions, weather, or time of day.

(6) Level 5: Full Driving Automation

The ADS is completely independent and capable of performing all driving tasks under all road, environmental, and traffic conditions that a human driver could manage. There are no ODD restrictions, and human occupants are strictly passengers, requiring no steering wheel or pedals.

While levels 0 through 2 keep the safety onus on the human, the transition to level 3 and above introduces significant complexities in perception and behavioral modelling. Specifically, when operating in environments with unpredictable or aggressive road actors, a Level 3/4 system must not only track trajectories but also predict anomalous driving behaviors to ensure safety before a critical handover is triggered.

According to the Statista numbers, the current implementation of Level 4 AV is around 1% and estimated to reach 5% by the year 2030 [2]. Majority of the AV is dominated by Levels 1-3. As the scope of implementing Level 4 and above AV is increasing year by year, this does not in any means define the absolute takeover of the human driven system. Therefore, the environment where human-driven vehicles and AVs coexist, termed as the hybrid traffic situation, is set in place, as shown in Fig. 1.1.

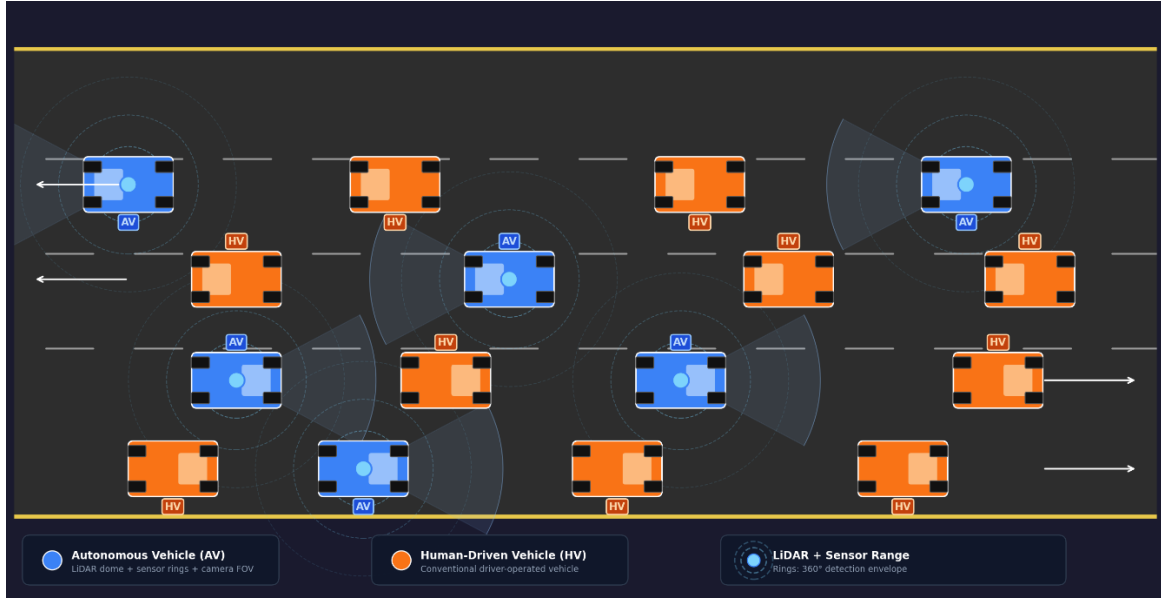


Fig 1.1 Hybrid traffic situation, constituting both AV and Human driven vehicles on road

1.2. Problem Statement

Given the suite of sensors available on a standard AV platform, this dissertation aims to build a predictive system that determines whether a nearby vehicle is about to perform a rash maneuver — using only data observable from the ego vehicle's own sensors. The system is strictly outward facing: it does not monitor the ego vehicle's own behavior, but instead anticipates external threats posed by surrounding traffic.

Throughout this dissertation, the term “rash driving” refers to a sudden, high-risk vehicle maneuver — such as an abrupt lane cut-in or emergency braking — that creates an immediate collision risk for nearby vehicles. This is distinct from “aggressive driving” as used in much of the existing literature, which typically characterizes a prolonged behavioral disposition (e.g., habitual speeding or tailgating) of the monitored vehicle itself. A rash event, as defined here, is a discrete and kinematically identifiable incident originating in a third-party vehicle and observable from the ego vehicle’s sensor perspective.

The existing body of research on aggressive driving detection has almost exclusively addressed ego-driver monitoring — detecting whether the person inside the AV is driving aggressively, using that vehicle's own IMU, GPS, and steering data. This includes fleet safety monitoring systems, smartphone-based driving behavior scoring applications, and in-cabin risk assessment tools. This dissertation takes a fundamentally different direction: rather than looking inward, the system looks outward, anticipating rash behavior in nearby third-party vehicles before or at the moment it occurs. We term this framing reactive threat anticipation. It is practically more important than ego-monitoring: an AV that can only monitor itself is reactive to its own mistakes, whereas an AV that can anticipate the mistakes of others can act pre-emptively to protect its occupants.

The sensor inputs considered in this work are: six-view surround RGB cameras, six-view semantic segmentation maps, ego-centric 360° LiDAR rendered as a Bird's-Eye View (BEV), and a kinematic annotation stream encoding the positions, speeds, and headings of the ego vehicle and up to three nearby vehicles. The prediction target is a binary label — rash or normal — assigned at each frame of a sliding temporal window.

The core challenge addressed by this dissertation is threefold. First, rash events are temporally sparse and brief, occurring over a span of one to two seconds, which constrains the amount of usable signal available before and during the event. Second, no publicly available dataset provides the combination of frame-precise labels and multi-modal sensor capture required for this task — real-world datasets either lack rash event labels entirely or provide only coarse, human-annotated timestamps. Third, the prediction system must generalize across road environments it has never encountered during training, a property that is difficult to validate using fixed-map simulation platforms or limited real-world recording locations.

This dissertation tries to tackle the issue of developing a system to aid the AV in predicting anomalous behavior in the hybrid traffic situation. The results aim to help the AVs to move

forward in the implementation of Level 3 and above at a higher pace than the predicted 5% in 2030.

1.3. Research Objectives

The objectives of this dissertation are listed as follows:

- (1) To construct a novel, purpose-built dataset for third-party rash driving prediction, with frame-precise binary event labels and simultaneous capture of four sensor modalities across procedurally varied road environments;
- (2) To establish whether kinematic motion data alone — analogous to GPS, IMU, and RADAR-derived tracking in a real AV — is sufficient to predict rash events in nearby vehicles;
- (3) To design and evaluate a multi-modal fusion architecture that combines spatial (LiDAR BEV), visual (RGB and semantic), and kinematic information through a temporal model, and to determine whether this fusion meaningfully improves on the kinematic-only baseline;
- (4) To evaluate the resulting system's ability to generalize road environments and vehicle trajectories never seen during training;
- (5) To assess the computational feasibility of the proposed system for deployment on constrained onboard AV hardware.

1.4. Significance of Study

This work addresses a problem that sits at the intersection of two well-studied but previously disconnected research areas: aggressive/rash driving detection, which has focused on the ego vehicle, and multi-modal sensor fusion, which focuses on per-frame spatial tasks such as object detection. By combining these threads into a single temporal prediction task framed around

third-party anticipation, this dissertation opens a research direction with direct relevance to AV safety system design — specifically, the design of pre-emptive collision-avoidance triggers that act before a dangerous maneuver is complete.

The dataset constructed for this work is, to the author's knowledge, the first to provide frame-precise binary rash event labels alongside four simultaneous sensor modalities in a simulation environment with procedurally varied road topology. This resource may be of independent value to future researchers studying related prediction or fusion tasks.

1.5. Organization of Dissertation

The remainder of this dissertation is organized as follows. Chapter 2 reviews the related literature across four areas — ego-centric aggressive driving detection, multi-modal sensor fusion, trajectory and behavior prediction, and surrounding vehicle intent recognition — and identifies the specific gap this work addresses. Chapter 3 describes the construction of the dataset, including platform selection, the data collection protocol, and sensor synchronization. Chapter 4 presents the proposed system architecture in detail, covering each modality's feature encoding and the fusion and temporal modelling strategy. Chapter 5 presents the experimental results for all three models, an ablation analysis, and a discussion of generalization and computational efficiency. Chapter 6 concludes the dissertation and outlines directions for future work.

Chapter 2

Literature Review

2.1. Ego Centric Aggressive Driving Detection

The predominant approach to aggressive driving detection in the current literature treats the problem as one of ego-vehicle behavior monitoring. Smartphone-based sensing platforms use accelerometers, gyroscopes, and GPS data to classify a vehicle's own maneuvers into smooth or aggressive categories [3], [4]. Decision-tree and Hidden Markov Model classifiers have been used to detect abnormal driving behavior from inertial sensor streams, with reported accuracy above 90% on multi-class abnormal driving classification tasks [5]. More recent work formalizes an Inertial Measurement Unit based (IMU-based) "aggressiveness index" for wearable-sensor-driven driver assessment [6].

These approaches share a fundamental limitation relative to the goals of this dissertation: they characterize the behavior of the monitored vehicle itself, not the vehicles around it. A driving behavior scoring system that determines the ego vehicle which brakes smoothly has no information about whether the vehicle ahead is about to brake suddenly. The ego-centric framing, while well-suited to fleet safety monitoring and insurance telematics applications, is structurally insufficient for the proactive AV safety problem this dissertation addresses.

2.2. Multi-Modal Sensor Fusion for Autonomous Vehicles

Sensor fusion has been extensively studied in the context of object detection and scene understanding; Huang et al. provide a comprehensive survey and novel taxonomy covering over 50 LiDAR-camera fusion methods across detection and segmentation tasks [7]. BEVFusion unifies surround-camera and LiDAR features in a shared Bird's-Eye View representation space, preserving both geometric and semantic information that point-level

fusion approaches discard; the authors report large accuracy gains on 3D detection and segmentation benchmarks alongside a substantial reduction in inference latency through optimized BEV pooling [8]. TransFuser fuses perspective-view camera features and LiDAR BEV maps through multi-resolution transformer self-attention for end-to-end imitation-learning-based driving policy learning, reporting a 76% reduction in collisions relative to geometry-based fusion baselines in the CARLA simulator [9]. PointPainting projects LiDAR points into the output of an image-based semantic segmentation network and appends the resulting class scores to each point before passing the "painted" point cloud to a LiDAR-only detector, improving the performance of multiple state-of-the-art detectors on the KITTI and nuScenes benchmarks [10], [11]. Complexer-YOLO further combines semantic point cloud representations — formed by projecting LiDAR points onto visual semantic segmentation outputs — with a real-time 3D detector and multi-target tracker, demonstrating that semantic enrichment of raw LiDAR data consistently improves 3D object detection accuracy [12].

These works establish that combining camera and LiDAR information in a shared spatial representation — particularly a Bird's-Eye View grid — yields substantial benefits over single-modality or naively concatenated approaches. This principle directly informs the design of the BEV-CNN branch used in this dissertation's proposed architecture. However, all three works address per-frame spatial tasks: object detection, semantic segmentation, or single-step control. None addresses temporal behavior prediction, and none uses multi-modal fusion to forecast the future behavior of a specific nearby vehicle over a sliding temporal window — the task this dissertation addresses.

2.3. Trajectory and Behavior Prediction

A separate body of work addresses the prediction of future spatial trajectories rather than discrete behavioral events. Social LSTM predicts future pedestrian positions by pooling the

hidden states of spatially nearby agents within an LSTM framework, capturing social interaction effects in a data-driven manner rather than through hand-crafted force functions [13]. Convolutional Social Pooling extends this idea to vehicle trajectory prediction on highways, replacing the fixed social pooling grid with convolutional layers that more robustly learn inter-vehicle dependencies from an LSTM encoder-decoder architecture [14].

These methods are valuable in establishing that the relative motion of nearby agents carries strong predictive signal for future behavior — a finding consistent with the strong performance of the Kinematic LSTM baseline in this dissertation (Chapter 5). However, trajectory prediction methods output continuous future positions rather than a discrete rash/normal classification, and they do not typically consume raw camera or LiDAR imagery directly, limiting their applicability to settings where only positional history is available.

2.4. Surrounding Vehicle Intent and Anomaly Recognition

The line of work closest in spirit to this dissertation addresses recognition of anomalous or risky intent in surrounding vehicles, rather than the ego vehicle, and therefore merits separate discussion from the general trajectory prediction literature.

A time-sequenced weighted Hidden Markov Model has been used to recognize the driving intention (e.g., lane-change direction) of nearby vehicles from trajectory data on the highD highway dataset, reporting classification accuracies above 93% for left and right lane changes [15]. RiskNet proposes an interaction-aware, field-theoretic risk forecasting framework that combines deterministic interaction modelling with a graph-neural-network-based trajectory predictor to assess risk posed by surrounding agents across highway, intersection, and roundabout scenarios, validated on the highD, inD, and rounD naturalistic driving datasets [16]. Most directly comparable to this dissertation, a 2025 framework (referred to here as ADID+) recognizes anomalous intent — including malicious cut-ins and illegal overtaking — in

surrounding vehicles using frequency-domain dual-stream attention over vehicle-road-cloud (V2X infrastructure) collaborative data, reporting F1 improvements of 6.5–18.6% over prior methods on a 12,586-sample multi-modal dataset covering six anomaly types [17].

This dissertation differs from this closest body of work in three respects. First, all three methods rely on trajectory data derived either from drone-captured overhead recordings (highD, inD, rounD) or from roadside/V2X infrastructure sensors — none uses the *ego AV's own onboard* camera, LiDAR, and kinematic sensor suite as the sole input, which is the deployment-relevant setting this dissertation targets. Second, none of these methods combines raw camera imagery, semantic segmentation, and LiDAR spatial (BEV) representations with kinematics within a single fusion model; their inputs remain trajectory- or infrastructure-data-centric. Third, none provides frame-precise binary event labels at the exact onset of a scripted rash maneuver — their anomaly windows are inferred post hoc from naturalistic recordings or human annotation rather than generated with sub-frame precision in a controlled simulation. To the author's knowledge, this dissertation is the first to address third-party rash event prediction purely from an ego AV's onboard multi-modal sensor suite, using a frame-precise, simulation-generated dataset.

2.5. Simulation Platforms for Autonomous Driving Research

The use of simulation for AV research has grown substantially since the release of CARLA, an open-source Unreal-Engine-based simulator that provides photorealistic rendering, twelve fixed named maps, and a client-server traffic manager for scripting non-player vehicle behavior [18]. As shown in Fig. 2.1, MetaDrive was developed specifically to support generalizable reinforcement learning research, using procedural generation to compose an effectively unlimited number of unique road topologies from both synthetic generation and real-world map

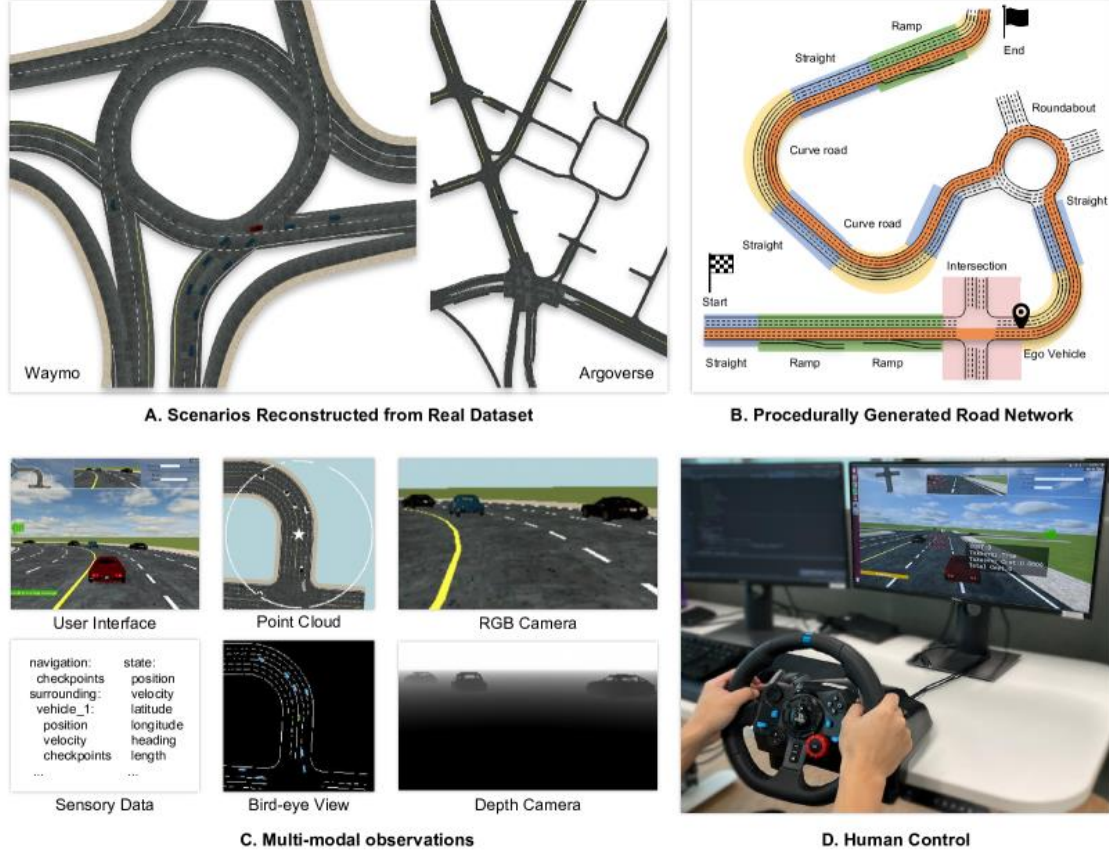


Fig 2.1 MetaDrive simulation platform [19]

data (Argoverse, Waymo, nuScenes), while running as a single lightweight, CPU-only Python process built on the Panda3D rendering engine [19].

Existing simulation-based datasets built on these platforms predominantly target normal driving, 3D object detection, or end-to-end control policy learning, and typically provide only a subset of sensor modalities relevant to any one downstream task. No existing simulation dataset combines frame-precise rash event labels, simultaneous four-modality capture (RGB, semantic segmentation, LiDAR BEV, kinematics), and scripted events with known NPC identity — the specific combination of properties this dissertation's dataset construction (Chapter 3) is designed to provide.

2.6. Summary and Identified Research Gap

Three distinct gaps emerge from this review. First, the aggressive driving detection literature is ego-centric and does not address third-party anticipation. Second, the multi-modal sensor fusion literature addresses per-frame spatial tasks rather than temporal third-party behavior prediction. Third, the closest existing work on surrounding vehicle anomaly recognition relies on infrastructure or overhead trajectory data rather than an ego AV's own onboard sensor suite and does not provide frame-precise simulation-generated labels. This dissertation addresses the intersection of these three gaps: a third-party rash prediction task, solved using a multi-modal onboard sensor fusion architecture, trained on a purpose-built, frame-precise simulation dataset.

Chapter 3

Dataset Construction

3.1. Simulation Platform Selection

The dataset is constructed using MetaDrive 0.4.3, a lightweight open-source AV simulation framework developed at UCSD and HKUST. MetaDrive's rendering and physics layer is built on Panda3D, an open-source, CPU-friendly 3D game engine (originally developed at Carnegie Mellon University and subsequently by Disney) that does not require dedicated GPU acceleration to render scenes or compute rigid-body physics. MetaDrive layers a procedural road-network generator on top of this engine: rather than loading a fixed, hand-authored map, each episode samples a numeric seed and algorithmically composes a unique sequence of roadblocks (straights, curves, intersections, ramps) into a drivable network, optionally drawing block layouts and traffic density parameters from real-world map data such as Argoverse, Waymo Open, and nuScenes. Background traffic in each episode is controlled by an Intelligent Driver Model (IDM) car-following policy, a longitudinal control model widely used in traffic simulation to produce realistic acceleration and deceleration behavior in response to a lead vehicle's speed and following distance; the scripted rash events used in this dissertation (Section 3.4) are implemented as scripted overrides layered on top of this baseline IDM behaviour for the triggering NPC.

The reason why MetaDrive was selected as the primary simulator over the other commonly used CARLA simulator on both technical and principled grounds is summarized in Table 3.1.

Table 3.1 MetaDrive vs. CARLA comparison

Criterion	MetaDrive 0.4.3	CARLA 0.9.x
GPU Requirement	None — CPU only	Dedicated GPU $\geq$ 8 GB VRAM
Installation	<code>'pip install metadrive-simulator'</code> (~200 MB)	~10 GB Unreal Engine binary
Runtime Architecture	Single Python process	UE4 server + Python client over TCP
Road Variety	Procedural generation, unlimited layouts	12 fixed named maps
Event Timestamp Precision	Frame-perfect (same process)	Subject to client-server jitter
Rendering Engine	Panda3D (lightweight)	Unreal Engine 4 (GPU-heavy)

Four arguments support this choice. First, photorealism does not improve feature quality in this work because all visual processing passes through a frozen MobileNetV3-Small backbone pretrained on ImageNet; since the backbone's weights do not update, the only factor relevant to feature quality is similarity to ImageNet images in feature space, which is governed by the pretraining itself rather than rendering fidelity. Second, fixed maps would harm the model's claim to generalization: if a limited number of training episodes shared the same twelve CARLA maps, the model could achieve high accuracy by recognizing map-specific visual patterns rather than learning general rash behavior cues. MetaDrive's procedural generation produces a structurally distinct road topology for every episode, making held-out test episodes genuinely novel environments. Third, frame-precise event labelling is architecturally simpler in MetaDrive: because NPC scripting and data recording execute in the same Python process and call stack, the event label is written at the exact simulation frame the rash behavior is triggered, with zero jitter — a property that would require careful client-server timestamp reconciliation in CARLA's networked architecture. Fourth, CPU-only operation reflects the

compute constraints of real onboard AV processors (e.g., NVIDIA Drive, Qualcomm Snapdragon Ride, Mobileye EyeQ), making a system that trains and runs without a dedicated GPU more directly relevant to eventual deployment.

For completeness, CARLA would be the preferred platform where sensor physical fidelity is itself a research variable (e.g., radar reflection physics), where pedestrian and cyclist behavior modelling is required, or where weather and lighting variation are experimental factors. None of these considerations apply to the research question addressed in this dissertation, which concerns fusion architecture and temporal prediction rather than sensor physics or environmental conditions.

3.2. Software and Hardware Environment

The complete dataset construction and model training pipeline was implemented and executed on a single Windows-based machine using the following software stack (Table 3.2):

Table 3.2: Software stack

Component	Version/Detail
Operating System	Windows 11
Python	3.8
Simulation Framework	MetaDrive 0.4.3 (<code>pip install metadrive-simulator</code>)
Rendering/physics backend	Panda3D (bundled with MetaDrive)
Deep Learning Framework	PyTorch 2.4.1, CPU build (no CUDA)
Visual Backbone	torchvision <code>'mobilenet_v3_small'</code> , ImageNet pretrained weights
Supporting Libraries	NumPy, OpenCV (<code>'cv2'</code>), scikit-learn, Matplotlib
Hardware	CPU only — no dedicated GPU used at any pipeline stage

Two environment-specific issues had to be resolved before reliable data collection was possible. First, MetaDrive's offscreen (headless) rendering path triggers a hang under the Windows WGL (OpenGL) driver binding when multiple camera sensors are attached; this was resolved by monkey-patching `metadrive.engine.engine_utils._fix_offscreen_rendering` to a no-op prior to environment construction, bypassing the problematic code path entirely. Second, because automation and scripting tooling used during development defaulted to a Linux/WSL shell while the project's Python interpreter and MetaDrive installation were Windows-native, data collection and training scripts had to be invoked through PowerShell using an explicit interpreter path (e.g., `& "C:\...\Python38\python.exe" -u script.py`) rather than a bare `python` command, to avoid silently resolving to the wrong Python environment.

All visual feature extraction, BEV CNN training, and BiGRU temporal modelling were performed on CPU throughout, with no part of the pipeline — from raw sensor collection through final model evaluation — requiring GPU acceleration. This is a deliberate consequence of the platform choice discussed in Section 3.2, not an incidental limitation.

3.3. Scene and Event Generation Protocol

Each episode runs the MetaDrive environment in headless mode for 60 seconds at 30 simulation frames per second, with the ego vehicle driven forward by a simple control policy. Non-player vehicles (NPCs) are scripted to trigger a rash event — either a sudden lane cut-in or an emergency brake — at a randomized time during the episode. When an event is triggered, the data recorder captures a sliding window of frames centered on the event: 30 frames (3 seconds) of pre-event buffer and 20 frames (2 seconds) of post-event buffer.

Frames are sampled at 10 FPS by recording every third simulation step, yielding a temporal resolution of 100ms per frame — consistent with the approximate reaction-time budget

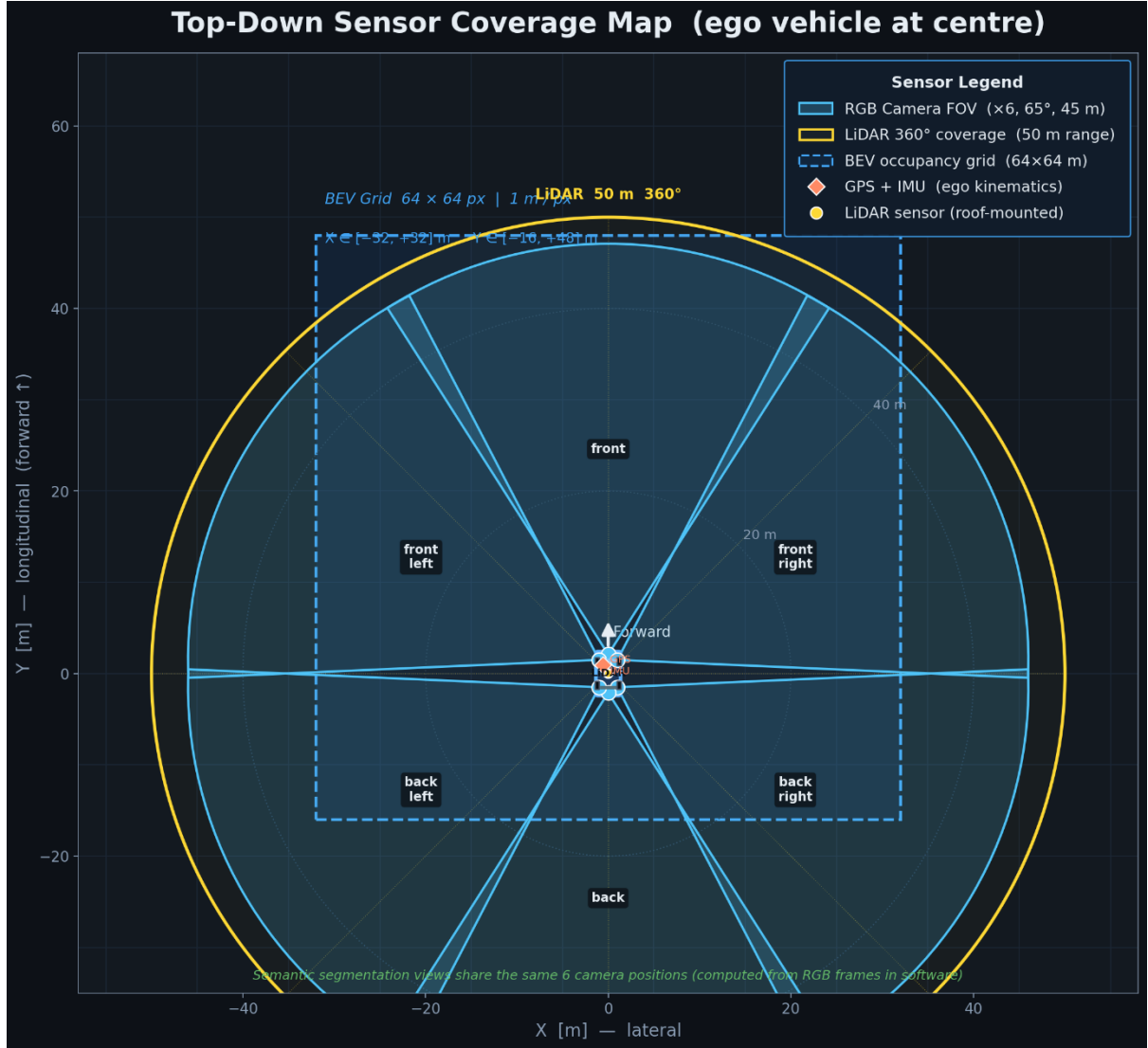


Fig 3.1. Top-Down sensor suite coverage

motivating this research. All four sensor modalities are captured synchronously at the same simulation step, ensuring perfect temporal alignment.

In an early data collection pass, the default NPC scripting cooldown and trigger probabilities were found to produce too few rash events per episode for efficient dataset construction — many 60-second episodes elapsed without a single triggered event. The NPC cooldown window was reduced (to 30–80 simulation steps) and trigger probabilities were raised accordingly, so that rash events occur reliably and early enough within each episode to make data collection tractable at the scale required (Section 3.7).

3.4. Engineering Corrections to the Sensor Pipeline

During development, a significant calibration defect was identified in MetaDrive's built-in LiDAR simulation. The simulator's native `PointCloudLidar.get_rgb_array_cpu()` method, used to convert a rendered depth buffer into a 3D point cloud, contains two compounding bugs: the horizontal and vertical principal-point intrinsics are swapped (the buffer height is used where the buffer width is expected, and vice versa), and the depth image is transposed prior to back-projection, misaligning each pixel with the wrong ray direction. The combined effect is a point cloud expressed in distorted, world-scale coordinates spanning roughly $-13,000\text{m}$ to $+18,000\text{m}$, rather than the expected ego-centric range of approximately $\pm 100\text{m}$ — rendering the native output unusable for this dissertation's ego-centric BEV construction (Section 3.5).

This defect was corrected by bypassing `get_rgb_array_cpu()` entirely. The raw normalized depth texture (`depth_tex`, float32 in the range $[0, 1]$) is accessed directly from each camera sensor, manually corrected camera intrinsics are applied, and 3D points are back-projected in Panda3D's native camera coordinate convention (Y-forward, X-right, Z-up) before the sensor's mount offset is applied to express each point in the ego vehicle's frame. This fix was verified by inspecting the resulting point cloud range against the expected ego-centric bounds prior to full-scale data collection.

Two related corrections were made to support accurate downstream geometric reasoning, though they are not directly consumed by the models in this dissertation. Bounding boxes for all detected objects are stored in both world frame and ego-centric frame, with the ego-centric transform computed by translating by the negative ego position and then rotating by the negative ego heading about the vertical axis; object box height is set using each object's true physical height rather than a placeholder value. A per-camera calibration file (`calibration.json`) is also generated for each episode, recording the 4×4 homogeneous

transformation matrices from each camera frame to the ego frame and from the LiDAR frame to each camera frame, in addition to the heading/pitch/roll mount offsets — enabling LiDAR points to be projected onto any camera image, a capability not exercised in the present fusion architecture but retained for future extensions of this dataset.

These corrections were necessary preconditions for the dataset to be usable for this dissertation's purposes: without them, the LiDAR-derived BEV grids described in Section 3.5 would have been computed from geometrically meaningless point coordinates, and the resulting BEV channel would have carried no genuine spatial signal.

3.5. Sensor Modalities and Synchronization

Surround RGB cameras: Six views at 640×360 pixels — front, front-left, front-right, rear, rear-left, and rear-right — provide full 360° visual coverage around the ego vehicle, as shown in Fig. 3.2.

Semantic segmentation: Six views at the same resolution and headings as the RGB cameras, with pixel-level class labels (road, vehicle, pedestrian, background), rendered natively by the MetaDrive/Panda3D pipeline, as presented in Fig. 3.3.

LiDAR, rendered as Bird's-Eye View: MetaDrive simulates LiDAR using depth-buffer rendering. For each of six headings (0° , 60° , 120° , 180° , 240° , 300°), a depth image is rendered and back-projected into 3D points; the six partial point clouds are concatenated into a single ego-centric 360° point cloud. This point cloud is then projected onto a 64×64 pixel Bird's-Eye View grid spanning 64m laterally (± 32 m) and 64m longitudinally (-16 m to $+48$ m) at 1m/pixel resolution, with the ego vehicle positioned at pixel (32, 16) — offset toward the rear of the grid so that more of the forward space (48m) is captured than the rear (16m), reflecting the forward-facing anticipation requirement of the task. The resulting three-channel grid encodes per-cell occupancy mean LiDAR intensity, and mean point height, as shown in Figs. 3.4 and 3.5.

Surround RGB Views -- Rash Event Frame (fid=00045)
Front camera at top-centre; back camera at bottom-centre

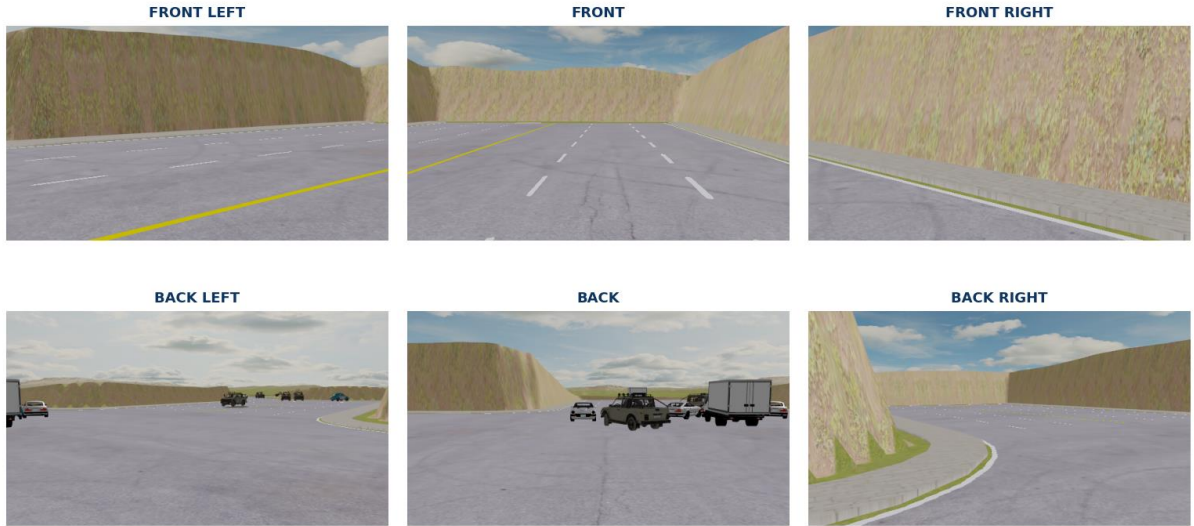


Fig 3.2. Surround camera images collected through the RGB camera modality.

Surround Semantic Segmentation Views -- same frame (fid=00045)

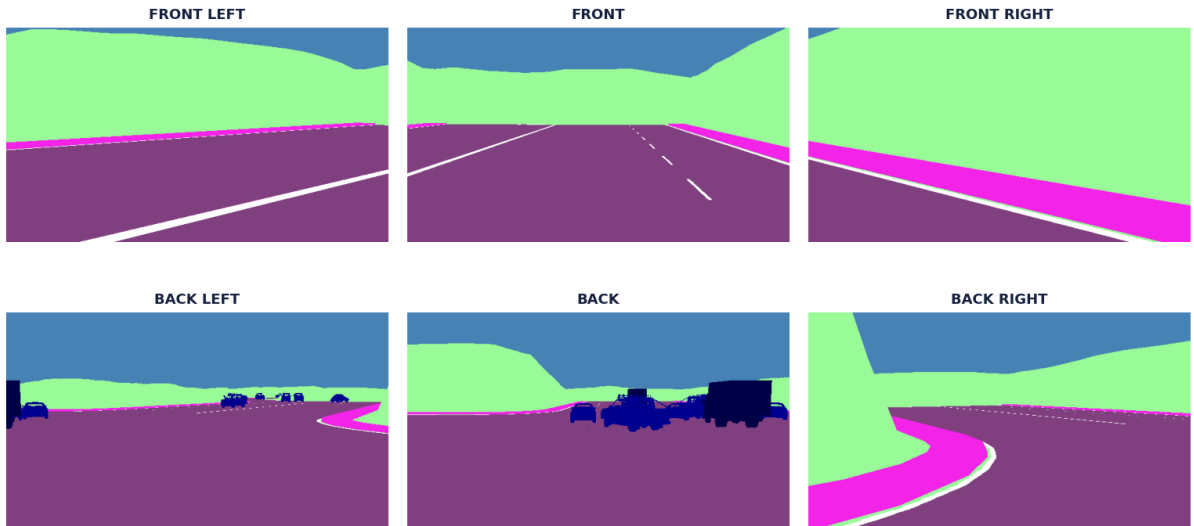


Fig 3.3. Surround camera images collected through semantic camera modality.

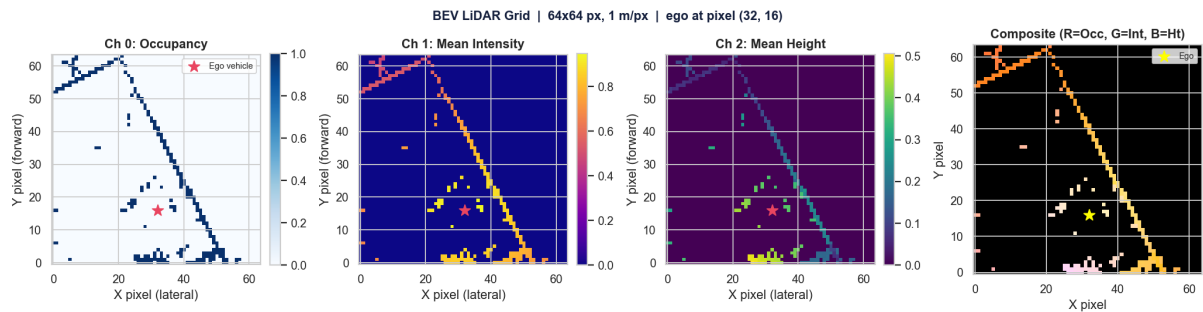


Fig 3.4. BEV LiDAR Grid 3 channel grid representation.

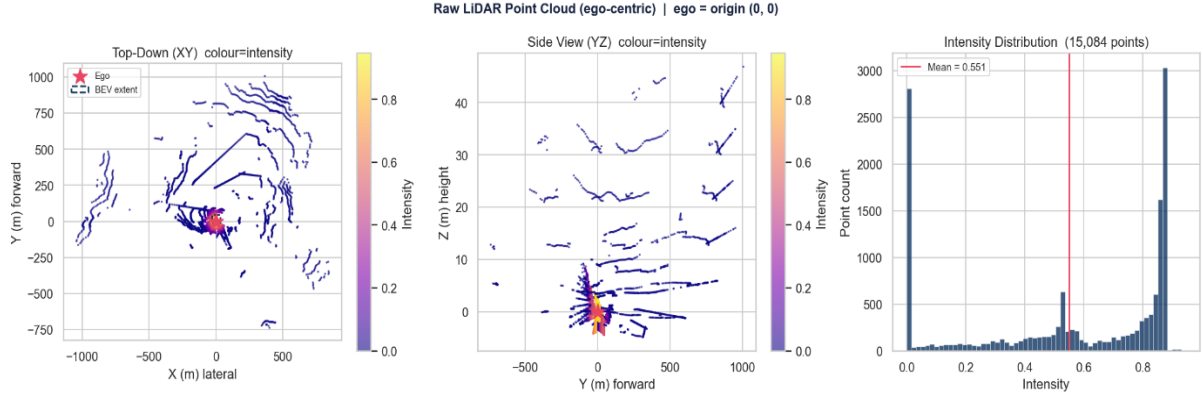


Fig 3.5. Raw LiDAR Point Cloud Representation.

Kinematic annotation: Each frame includes a structured JSON annotation recording the ego vehicle's state (speed in m/s, heading in degrees, x/y position in meters) and the states of up to three nearest NPCs (relative x/y position, speed, heading, distance). These are assembled into a 19-dimensional feature vector (Table 3.3). In a real AV, this information would be derived from GPS, IMU, wheel encoders, and RADAR/LiDAR-based object tracking with Kalman filtering; the simulator's JSON output is treated as a stand-in for this real-sensor-derived information, not as an additional idealized sensor.

Table 3.3 Kinematic feature vector composition

Dimensions	Description
0	Ego speed (m/s)
1	Ego heading (degrees)
2-3	Ego position (x, y)
4-8	NPC 1: relative x, relative y, speed, heading, distance
9-13	NPC 2: same fields (zero-filled if absent)
14-18	NPC 3: same fields (zero-filled if absent)

3.6. Label Schema

The binary label `'is_event_frame'` is set to True for every frame within the captured event window (the 3-second pre-buffer and 2-second post-buffer surrounding the scripted trigger). This labelling design reflects the nature of the prediction task: the model is required to learn the sensor patterns associated with an imminent or unfolding rash event, rather than to recognize only the single instant of peak danger.

3.7. Dataset Statistics and Train/Validation/Test Split

Fifty episodes were collected in total, of which approximately 44 yielded a valid rash event, producing 68 event clips and 17,559 frames. The resulting class distribution is heavily imbalanced: 14,727 rash-event frames (83.9%) versus 2,832 normal frames (16.1%). This imbalance is addressed during model training through class-weighted loss (Chapter 4).

The dataset is partitioned at the episode level, not the frame level, to prevent data leakage. Test episodes {9, 10, 47, 49} yield 6 clips (~1,697 frames); validation episodes {3, 6, 45, 46} yield 5 clips; all remaining episodes (57 clips, ~14,400 frames) form the training set. An episode-level split is essential to this dissertation's generalization claims: frames drawn from the same episode share road layout, NPC trajectory, and background scene geometry, and a frame-level split would permit nearly identical sensor readings to appear in both the training and test sets, artificially inflating reported performance.

The raw dataset (RGB, semantic, LiDAR, and JSON annotations for all 50 episodes) occupies approximately 27 GB of disk storage. A second, much smaller derived artefact — the cached backbone features and BEV grids described in Sections 4.3 — occupies approximately 527 MB, a reduction made possible by caching only the compact outputs of the frozen visual backbone and float16 BEV grids rather than retaining the model-ready tensors of the raw imagery.

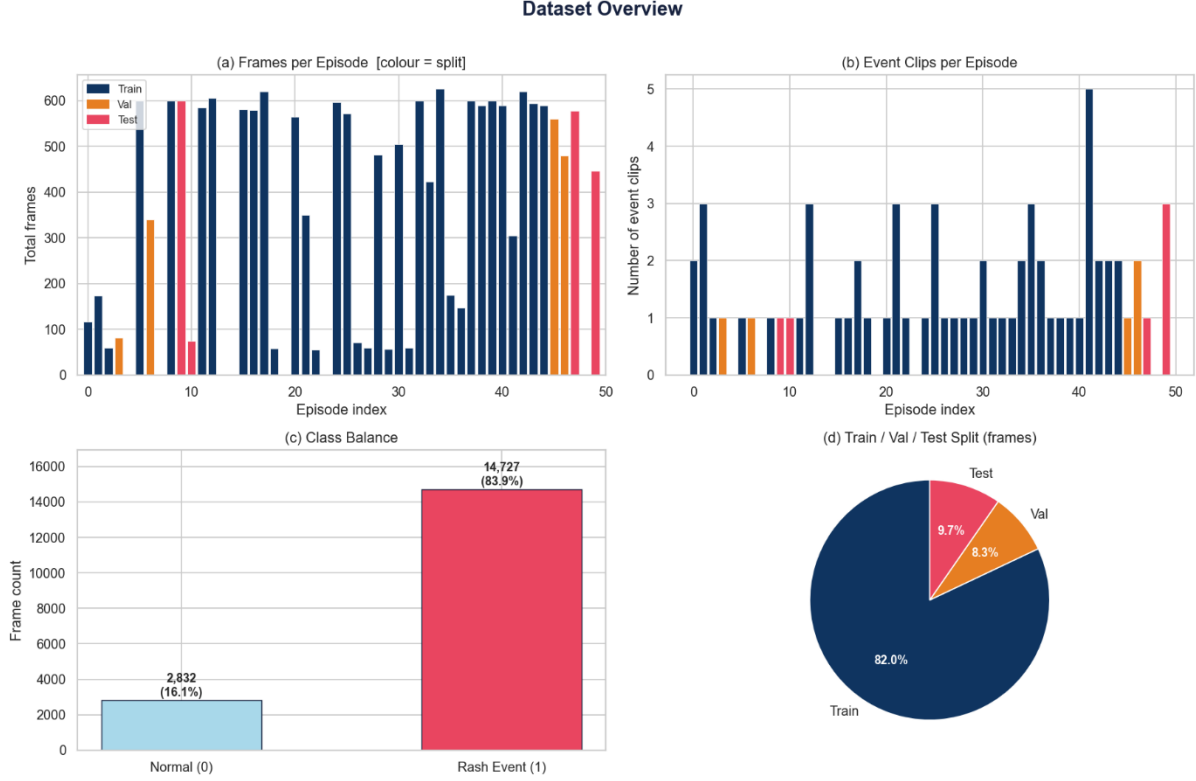


Fig 3.6. Dataset overview visualization

3.8. Reproducibility

The dataset and all downstream models can be reproduced from a clean environment using the software stack listed in Table 3.2. The end-to-end procedure is as follows.

- (1) **Collect the raw dataset:** Run ``collect_dataset.py``, which drives the MetaDrive environment through repeated episodes and writes RGB, semantic, LiDAR, and JSON annotation data for each triggered event to ``metadrive_fusion_dataset/``. This step is long-running (many hours) and is stopped once a target number of episodes has been collected; 50 episodes were used in this dissertation.
- (2) **Build the feature cache:** Run ``train_early_fusion_v2.py``, which automatically detects an incomplete ``feature_cache_v2/`` directory and executes a Phase 1 caching pass — batched MobileNetV3-Small inference over all RGB and semantic frames, together with BEV grid construction — before proceeding to training. This phase took approximately 46 minutes for all 17,559 frames in this dissertation's dataset.

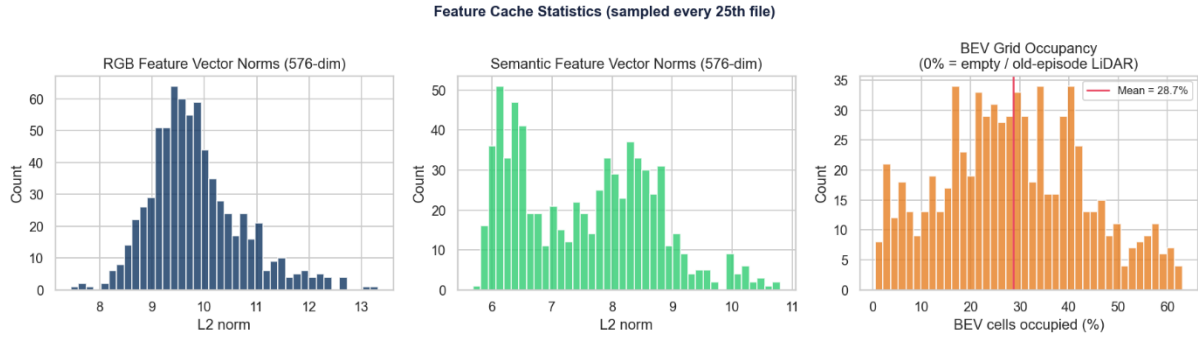


Fig 3.7. Feature caching statistics for the dataset

- (3) **Train the three models:** Run ``train_kinematic_lstm.py`` (Model 1), ``train_early_fusion.py`` (Model 2), and ``train_early_fusion_v2.py`` (Model 3) in turn; each script trains its respective model and writes checkpoints, training logs, and diagnostic plots to ``models/``.
- (4) **Evaluate on the held-out test set:** Run ``eval_ef2.py``, which loads only the validation and test episodes (avoiding the memory cost of loading all 57 training clips), sweeps the decision threshold, and reports F1, precision, recall, AUROC, and the full classification report (Chapter 5).
- (5) **Verify data loading performance:** Run ``test_load.py``, a diagnostic script that reports per-modality memory usage and load time, used to confirm that the dataset fits within the memory budget of the CPU-only training environment described in Section 3.2.

3.9. Summary

This chapter has described the construction of a dataset purpose-built for third-party rash driving prediction: 50 procedurally generated episodes, frame-precise binary labels, and four synchronized sensor modalities, collected using a corrected and calibrated sensor pipeline within a fully CPU-only, reproducible software environment, and partitioned at the episode level to support genuine generalization claims. Chapter 4 describes the system architecture designed to consume this dataset.

Chapter 4

Proposed System Architecture

4.1. Overview

This chapter presents the architecture of Early Fusion V2, the primary system proposed in this dissertation, along with the two preceding models (Kinematic LSTM and Early Fusion V1) that motivate its design. Early Fusion V2 processes four sensor streams in parallel per frame, projects each into a compact feature embedding, concatenates these embeddings into a unified per-frame representation, and passes the resulting sequence through a Bidirectional GRU to capture temporal dynamics across the sliding event window. A classification head then maps the final GRU hidden state to a binary rash/normal prediction.

The architecture is designed around three guiding principles. First, each modality should contribute complementary information that the others cannot provide on their own. Second, the temporal model should reason bidirectionally over the event window, since the rash behavior unfolds gradually rather than instantaneously. Third, the system must remain trainable and deployable on CPU hardware without a dedicated GPU, consistent with the deployment motivation discussed in the Introduction section.

4.2. Design Rationale: Why Feature-Level (Early) Fusion

Early (feature-level) fusion is chosen over late fusion for this architecture. Concatenating all modality embeddings before temporal modelling allows the Bidirectional GRU to learn cross-modal temporal interactions — for example, correlating a sudden change in kinematic distance with a corresponding shift in BEV occupancy and a simultaneous change in RGB appearance. A late-fusion design, in which each modality's temporal sequence is modelled independently

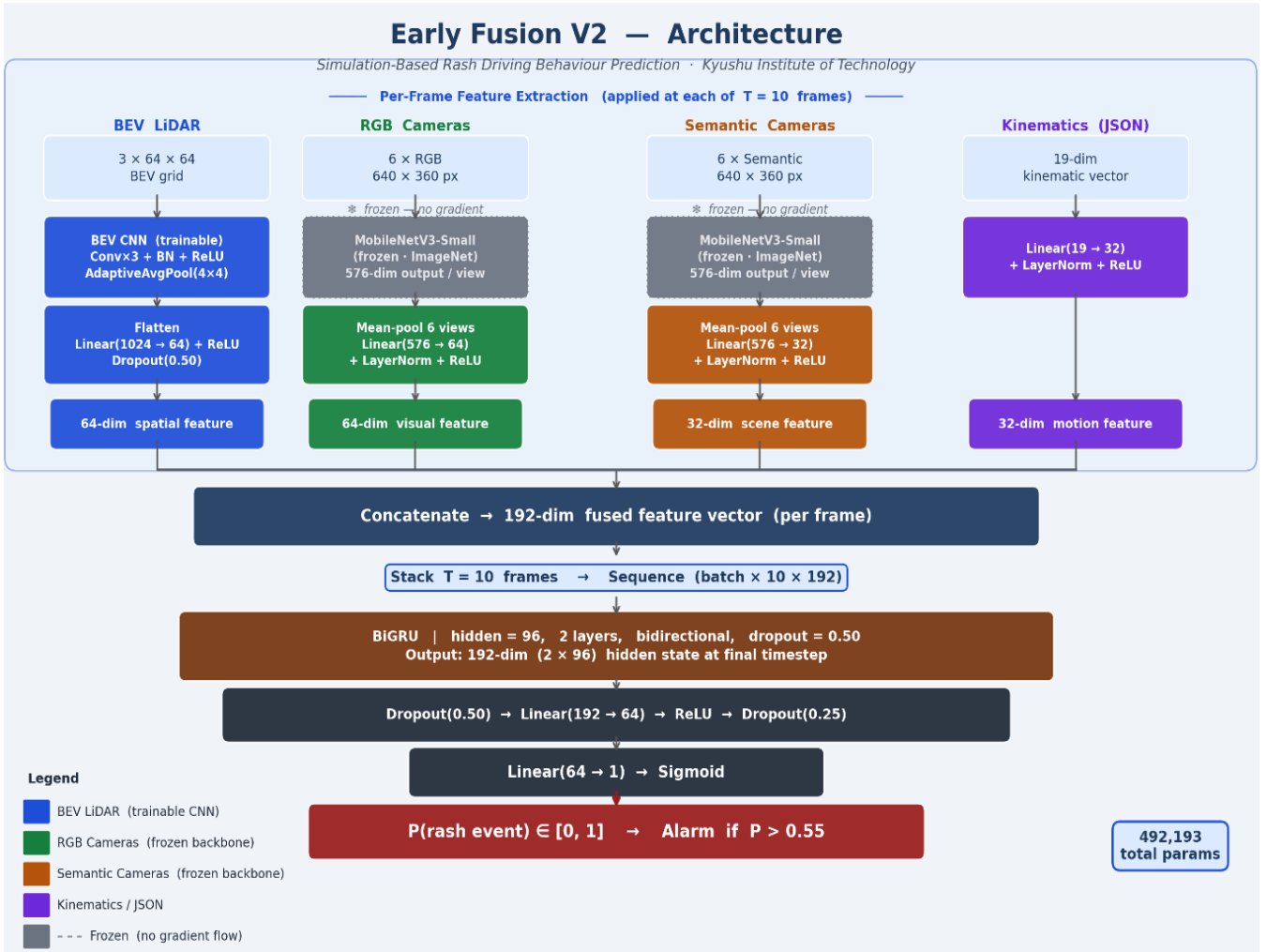


Fig 4.1. Early Fusion V2 architecture diagram. Four modality-specific encoders (BEV LiDAR, RGB cameras, semantic cameras, kinematics) feed a 192-dimensional fused feature vector into a Bidirectional GRU and classification head. Trainable components (BEV CNN, projection heads, GRU, classification head) and frozen components (MobileNetV3-Small backbones) are color-coded separately; total model size is 492,193 parameters.

before combining final predictions, would prevent the model from learning these cross-modal temporal correlations directly.

4.3. Model Architecture

Fig. 4.1 summarizes the full Early Fusion V2 pipeline. Each frame is processed by four parallel branches. The kinematic branch projects the 19-dimensional motion vector through a

small trainable head (Linear, LayerNorm, ReLU) to a 32-dimensional embedding. The RGB and semantic branches each pass their six camera views through a shared, frozen MobileNetV3-Small backbone (pretrained on ImageNet, kept frozen to avoid backpropagating through 1.5 million parameters on CPU hardware and to preserve a feature space anchored to real-world visual statistics), mean-pool the resulting 576-dimensional features across views, and project them to 64 and 32 dimensions respectively. The BEV branch passes the three-channel, 64×64 occupancy grid described in Chapter 3 through the only spatial encoder trained entirely from scratch — a three-layer convolutional network (~89,000 parameters) — to a 64-dimensional embedding. These four embeddings are concatenated into a 192-dimensional per-frame vector.

Formally, let $G_t \in \mathbb{R}^{3 \times 64 \times 64}$ denote the BEV occupancy grid, $\{I_t^c\}_{c=1}^6$ the six RGB camera views, $\{S_t^c\}_{c=1}^6$ the semantic maps, $k_t \in \mathbb{R}^{19}$ the kinematic vector, and $\varphi(\cdot)$ the frozen MobileNetV3-Small backbone. The four encoder outputs per frame t are:

$$f_{bev,t} = BEV - CNN(G_t) \in \mathbb{R}^{64} \quad (4.1)$$

$$f_{rgb,t} = W_{rgb} \cdot \frac{1}{6} \sum_{c=1}^6 \varphi(I_t^c) \in \mathbb{R}^{64} \quad (4.2)$$

$$f_{sem,t} = W_{sem} \cdot \frac{1}{6} \sum_{c=1}^6 \varphi(S_t^c) \in \mathbb{R}^{32} \quad (4.3)$$

$$f_{kin,t} = ReLU(LN(W_{kin} \cdot k_t)) \in \mathbb{R}^{32} \quad (4.4)$$

where W denotes trainable projection matrices, LN Layer Normalisation, and φ the frozen MobileNetV3-Small backbone shared across camera streams. These are concatenated into the 192-dimensional per-frame fused vector, as in Equation 4.5 [20]:

$$z_t = [f_{bev,t}; f_{rgb,t}; f_{sem,t}; f_{kin,t}] \in \mathbb{R}^{192} \quad (4.5)$$

The sequence $Z = [z_1, z_2, \dots, z_T] \in \mathbb{R}^{T \times 192}$, $T = 10$, is passed through a two-layer Bidirectional GRU (hidden size $h = 96$ per direction). The forward and backward hidden states at the final step are concatenated to give $h_T \in \mathbb{R}^{192}$, which is then mapped to a scalar rash-event probability through the classification head. The final prediction uses a calibrated threshold $\theta = 0.55$ (Equation 4.6):

$$\hat{p} = \sigma(\text{classifier}(h_T)), \quad \hat{y} = \mathbb{1}[\hat{p} > \theta], \quad \theta = 0.55 \quad (4.6)$$

The resulting sequence of ten frames (one second of temporal context) is passed to a two-layer Bidirectional GRU (hidden size 96, dropout 0.5), which reasons about the event window in both directions so that later frames (e.g., the NPC's trajectory after initiating a cut-in) can help disambiguate earlier, more ambiguous ones. The final timestep's 192-dimensional hidden state is passed through a classification head (Dropout, Linear 192→64, ReLU, Dropout, Linear 64→1, Sigmoid) to produce the rash-event probability.

4.4. Training Configuration

The model is trained with binary cross-entropy loss, with class weighting applied to address the 83.9%/16.1% class imbalance described in Chapter 3.

The class weights are set proportional to the inverse class frequency: $w_+ = \frac{N}{2N_+}$ and $w_- = \frac{N}{2N_-}$, where $N_+ = 14,727$ (rash) and $N_- = 2,832$ (normal) are the training-set class counts. The weighted binary cross-entropy loss is given in Equation 4.7:

$$\mathcal{L} = -\frac{1}{N} \sum_i i \left(w_+ y_i \log(\hat{p}_i) + w_- (1 - y_i) \log(1 - \hat{p}_i) \right) \quad (4.7)$$

with an initial learning rate of 3×10^{-4} and weight decay of 5×10^{-4} , with cosine annealing learning rate scheduling over a maximum of 40 epochs as seen in Equation 4.8 and early stopping at a patience of 12 epochs.

$$\gamma_T = \frac{1}{2}(3 \times 10^{-4}) \left(i + \cos \left(\frac{T}{40} \pi \right) \right) \quad (4.8)$$

Dropout of 0.5 is applied within the GRU's inter-layer connections, at the output of the BEV CNN, and within the classification head, to counteract the overfitting tendency discussed in Chapter 5.

4.5. Comparison Models

Two additional models preceded Early Fusion V2 in this dissertation's experimental progression, motivating its design through direct comparison.

Kinematic LSTM processes only the 19-dimensional kinematic vector through a two-layer Bidirectional GRU (hidden size 64, approximately 184,000 parameters), testing whether motion data alone is sufficient for rash prediction.

Early Fusion V1 augments the kinematic branch with MobileNetV3 features from the six RGB camera views and a 9-dimensional statistical summary of the LiDAR point cloud (point count, mean and standard deviation of intensity, minimum/maximum/mean height, and intensity percentiles), testing whether adding visual and LiDAR information — in a simplified statistical form — improves on the kinematic baseline.

The results of all three models, and the architectural reasoning that motivated the progression from V1 to V2, are presented in Chapter 5.

4.6. Summary

This chapter presented the design of Early Fusion V2: four modality-specific encoders (kinematic, RGB, semantic, BEV), a feature-level fusion strategy, a Bidirectional GRU for temporal reasoning, and a lightweight classification head, all designed to remain trainable and deployable on CPU hardware. Chapter 5 evaluates this architecture against the two preceding models and presents a detailed results analysis.

Chapter 5

Experiments and Results

5.1. Overview

This chapter presents the experimental evaluation of the three models described in Chapter 4. It covers implementation details, evaluation metrics, the individual results of each model, an ablation analysis across the model progression, an analysis of overfitting behavior observed during training, an assessment of generalization to unseen road layouts, and a discussion of computational efficiency.

5.2. Implementation Details

All models are implemented in PyTorch 2.4.1 and trained using Python 3.8 on CPU hardware (no GPU). The MobileNetV3-Small backbone is loaded from `'torchvision.models'` with pretrained ImageNet weights and frozen prior to training. Visual features and BEV grids are precomputed and cached to disk in a Phase 1 extraction step, completed in approximately 46 minutes for all 17,559 frames, allowing subsequent training epochs to load cached features directly rather than repeating backbone inference.

A key implementation optimization in this phase was batching rather than processing each of the twelve per-frame images (six RGB plus six semantic) through the backbone sequentially, all twelve are stacked into a single batch tensor and processed in one forward pass. This reduced per-frame backbone processing time from approximately 96ms (12 sequential passes at ~8ms each, ~10 frames/second) to approximately 18ms (one batched pass, ~55 frames/second) a fivefold speedup that reduced total Phase 1 processing time from an estimated 2.5 hours to 46 minutes.

A second implementation consideration concerned memory usage. Storing all BEV grids in float32 precision would require approximately 860 MB ($17,559 \text{ frames} \times 3 \text{ channels} \times 64 \times 64 \text{ pixels} \times 4 \text{ bytes}$) and combined with cached RGB and semantic features and sliding-window sample copies, total memory usage was estimated at approximately 1.9 GB, exceeding available RAM during data loading. Storing BEV grids in float16 precision instead — converting to float32 only at the point of use within the data loader, at a cost of approximately 0.1ms per sample with no additional disk I/O — reduced total memory usage to approximately 514 MB, resolving the issue.

All sliding windows use a sequence length of ten frames (one second of temporal context), and training uses a batch size of 32. The optimal classification threshold for each model is calibrated by scanning the validation set before being applied to the held-out test set.

5.3. Evaluation Metrics

The primary evaluation metric used throughout this chapter is the F1 score on the rash-event class, selected because the dataset is class-imbalanced and because both false positives (triggering unnecessary corrective AV responses) and false negatives (failing to detect an actual rash event) carry meaningful practical cost. Secondary metrics reported are the Area Under the Receiver Operating Characteristic curve (AUROC), precision, and recall. All metrics in this chapter are reported on the held-out test episodes, which — as described in Chapter 3 — share no road layout or NPC trajectory with the training set.

The four metrics are defined in terms of the binary confusion matrix counts: true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN). Precision (Eq. 5.1) measures the fraction of positive predictions that are correct; Recall (Eq. 5.2) measures the fraction of actual positive instances correctly identified; F1 (Eq. 5.3) is the harmonic mean of

the two. AUROC is the area under the curve of Recall versus False Positive Rate $FPR = FP/(FP + TN)$ as the decision threshold θ is swept from 0 to 1.

$$Precision = \frac{TP}{TP + FP} \quad (5.1)$$

$$Recall = \frac{TP}{TP + FN} \quad (5.2)$$

$$F_1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall} = \frac{2TP}{2TP + FP + FN} \quad (5.3)$$

5.4. Model 1: Kinematic LSTM Baseline

The Kinematic LSTM model establishes whether kinematic motion data alone is sufficient for rash event prediction; if so, the more complex visual and spatial modalities introduced in later models would offer limited additional value. As shown in Figs. 5.1-5.3, training converges at epoch 4, with the best validation F1 of 0.9275. At a calibrated decision threshold of 0.10, the held-out test set yields an F1 of 0.9450, an AUROC of 0.9088, a precision of 0.9347, and a recall of 0.9556.

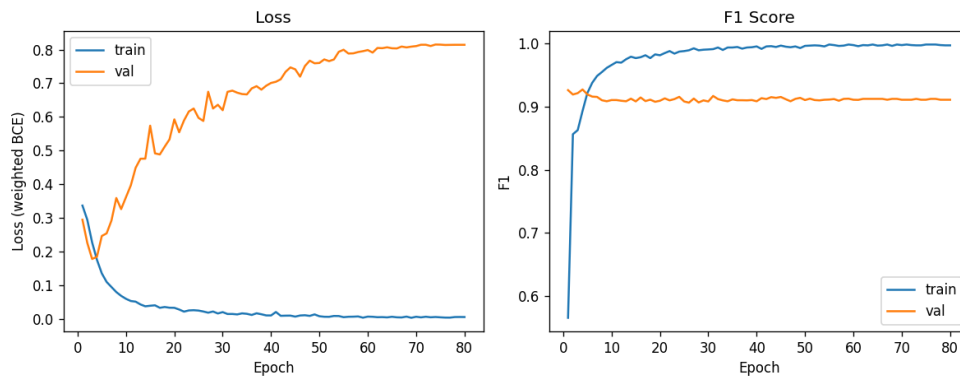


Fig 5.1. Training curve and the F1 score of Kinematic LSTM Model

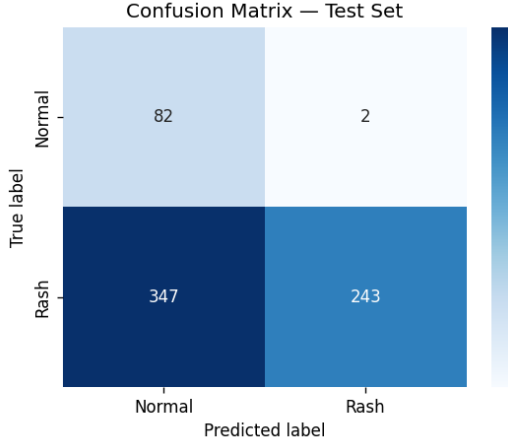


Fig 5.2. Confusion matrix

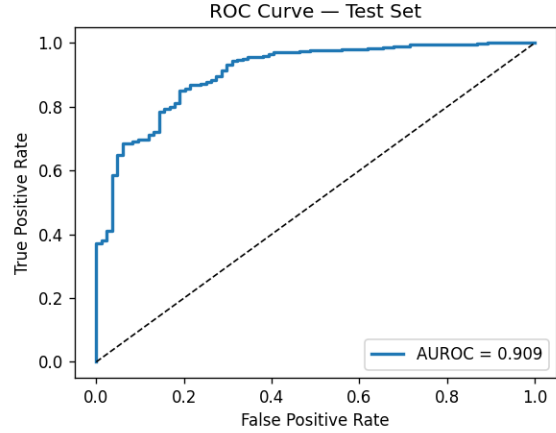


Fig 5.3. ROC curve

The unusually low calibrated threshold (0.10) arises from the use of a positive-class loss weight of 0.335 to address class imbalance during training, which biases the model's raw output probabilities downward; threshold calibration on the validation set corrects for this systematically. The strong baseline performance of this model is a central empirical finding of this dissertation: it indicates that the relative motion of nearby vehicles — speed differential, approach rate, and relative position — already carries most of the predictive signal required for rash event anticipation, even without any visual or spatial sensor information.

5.5. Model 2: Early Fusion V1

Early Fusion V1 augments the kinematic branch with MobileNetV3 features extracted from the six RGB camera views and a 9-dimensional statistical summary of the LiDAR point cloud. The three branches are projected and concatenated before being passed to a two-layer Bidirectional GRU (hidden size 64, approximately 185,000 parameters total).

As shown in Figs. 5.4-5.6, this model achieves a best validation F1 of 0.9265 at epoch 48 — statistically indistinguishable from the kinematic-only baseline's 0.9275. On the held-out test

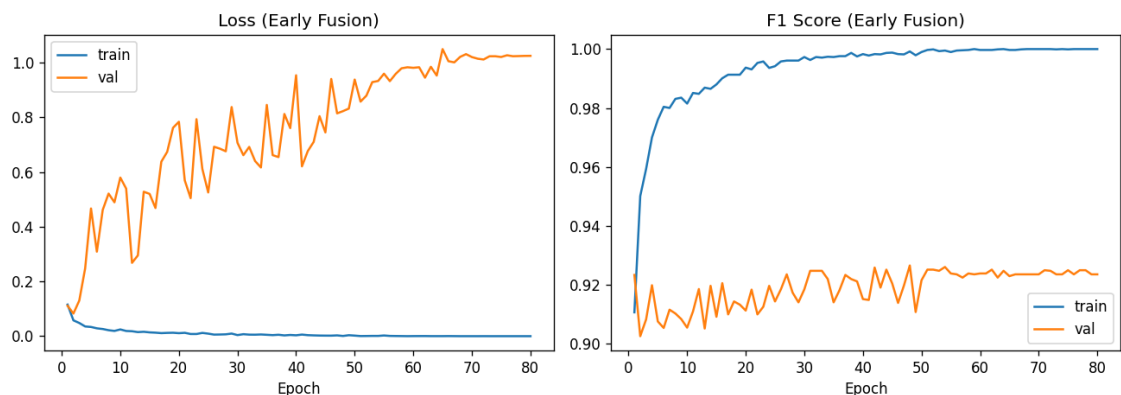


Fig 5.4. Training curve and F1 Score of Early Version V1

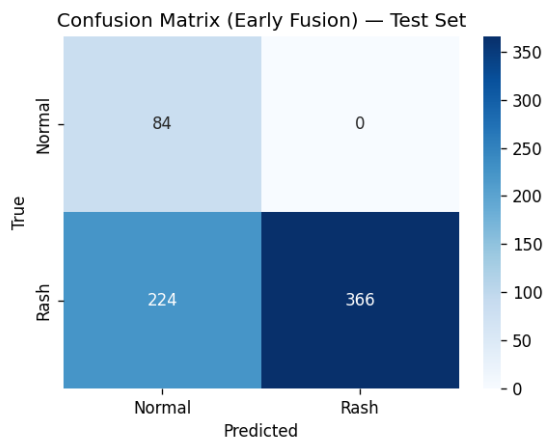


Fig 5.5. Confusion matrix

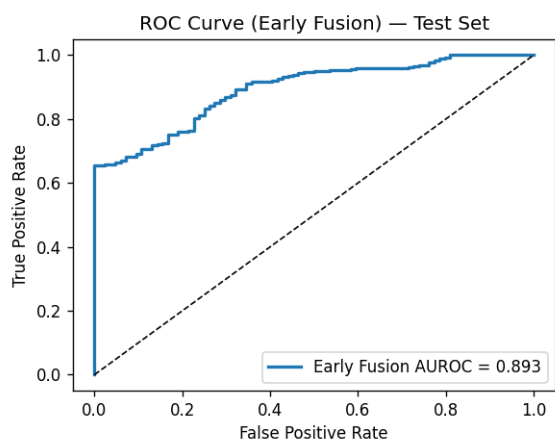


Fig 5.6. ROC curve

set, performance is markedly worse: F1 falls to 0.7657 at a calibrated threshold of 0.90, with an AUROC of 0.8927.

This result is an important negative finding for this dissertation's architectural reasoning. The near-identical validation F1 to the kinematic baseline indicates that adding RGB camera features and a statistical LiDAR summary provided no measurable improvement over motion data alone. Two factors are implicated. First, summarizing the LiDAR point cloud into nine scalar statistics discards essentially all spatial structure — information about the specific positions and shapes of nearby objects is lost. Second, the RGB branch, processed through a generic ImageNet-pretrained backbone without any spatial pooling tailored to the driving

domain, may introduce noise rather than signal, since ImageNet features are not designed to encode the specific proximity and approach-angle cues relevant to rash event prediction.

5.6. Model 3: Early Fusion V2 (Proposed Model)

Early Fusion V2 addresses the limitations identified in Early Fusion V1 through three changes: replacing the 9-dimensional LiDAR statistical summary with the full $64 \times 64 \times 3$ BEV grid processed by a trainable CNN (Chapter 4, Section 4.3); adding a dedicated semantic segmentation branch for scene understanding; and increasing the Bidirectional GRU hidden size from 64 to 96. The full 50-episode, 17,559-frame dataset is used for training, compared to the smaller 5,672-frame subset used in V1.

As shown in Figs. 5.7-5.9, this model achieves the best validation F1 across all three models, 0.9455, at epoch 2. The optimal decision threshold, calibrated on the validation set, is 0.55. At this threshold, the held-out test set yields an F1 of 0.9003, an AUROC of 0.8976, a precision of 0.9368, and a recall of 0.8665. The complete test-set classification report is shown in Table 5.1.

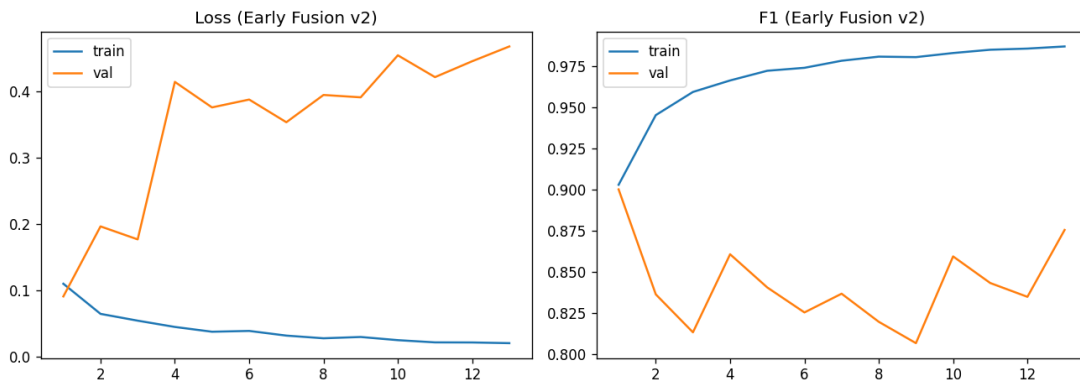


Fig 5.7. Training curve and F1 score of Primary Early Version V2

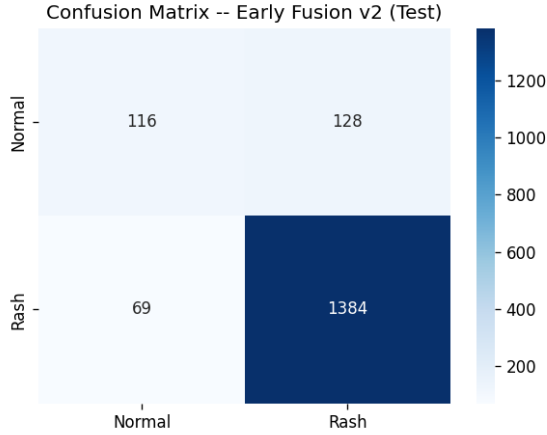


Fig 5.8. Confusion matrix

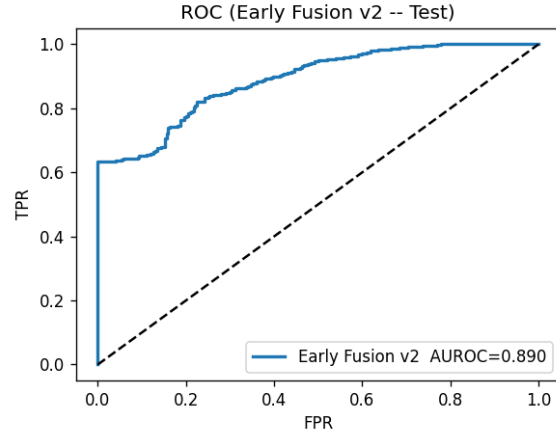


Fig 5.9. ROC curve

Table 5.1 Early Fusion V2 test-set classification report

Class	Precision	Recall	F1-Score	Support
Normal	0.45	0.65	0.53	244
Rash Event	0.94	0.87	0.90	1,453
Accuracy	-	-	0.84	1,697
Weighted Average	0.87	0.84	0.85	1,697

A precision of 0.9368 on the rash-event class indicates that 93.7% of the model's positive predictions correspond to genuine rash events, implying a low false-alarm rate — an important property for a system intended to trigger corrective AV responses. A recall of 0.8665 indicates the model correctly identifies 86.7% of all actual rash-event frames, missing 13.3%. The model's substantially weaker performance on the normal class (precision 0.45, recall 0.65) is attributable to the test set's class imbalance — only 244 normal frames versus 1,453 rash-event frames — and to the visual and kinematic similarity between normal frames and the quieter portions of event clips, which biases the model toward predicting "event."

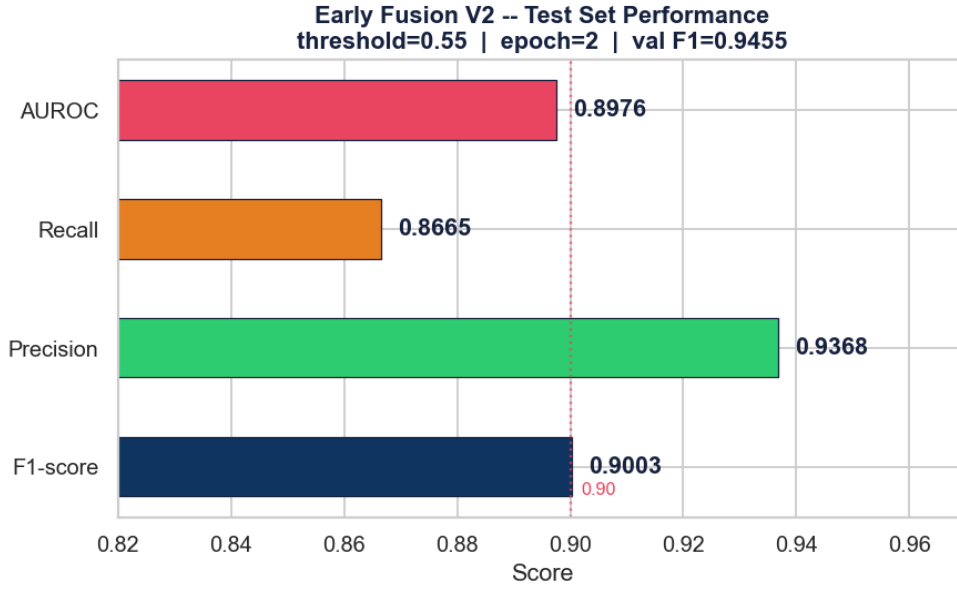


Fig 5.10. The proposed model's metric headline scores.

5.7. Ablation Study

The progression across the three models constitutes an implicit ablation over sensor modalities and architectural design choices, summarized in Table 5.2.

Table 5.2: Model comparison summary

Model	Key Modalities/Components	Validation F1	Test F1
Kinematics LSTM	Kinematics only	0.9275	0.9450
Early Fusion V1	+ RGB (MobileNetV3) + LiDAR statistics	0.9265	0.7657
Early Fusion V2	+ LiDAR BEV CNN + Semantic segmentation	0.9455	0.9003

The transition from V1 to V2 isolates the specific contribution of spatial LiDAR representation: replacing a 9-dimensional statistical summary with a CNN-processed BEV grid recovers — and exceeds — the test performance achieved by the kinematic-only baseline, while also pushing validation F1 to the highest value observed across all three models. This

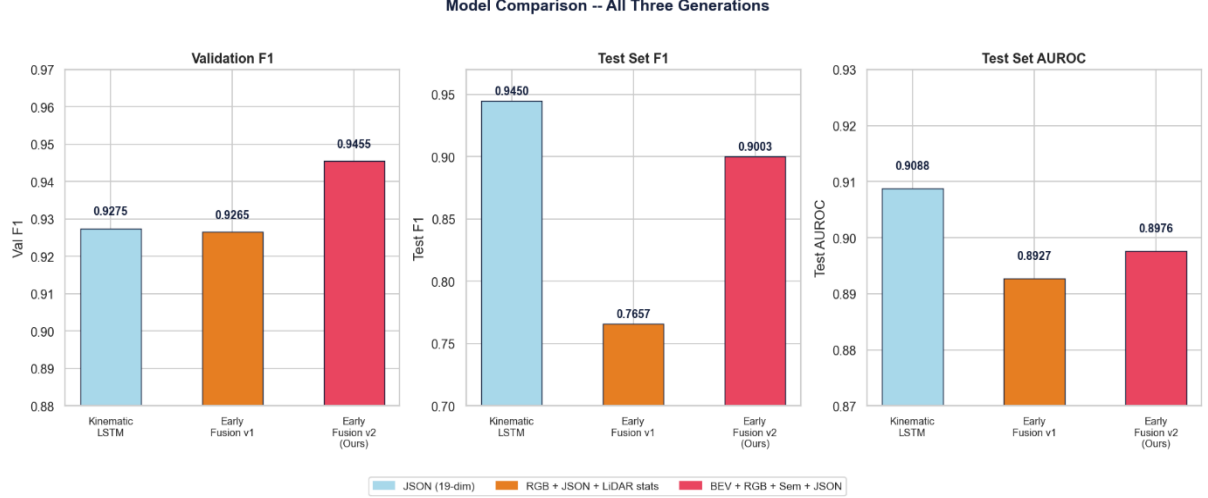


Fig 5.11. Ablation Comparison of the three Models – Kinematic LSTM, Early Fusion V1, Early Fusion V2 – respectively.

indicates that the value of LiDAR information for this task is contingent on preserving spatial structure; statistical summarization is insufficient. The addition of the semantic segmentation branch in V2 contributes complementary scene understanding — lane markings, vehicle boundaries — which is plausibly responsible for the model's improved generalization to the unseen road layouts in the test set, discussed further in Section 5.9.

5.8. Overfitting Analysis

Early Fusion V2 exhibits a pronounced and rapid overfitting pattern. An earlier training run using lower regularization (dropout 0.35, weight decay 1×10^{-4}) over 80 epochs showed training F1 climbing from 0.740 at epoch 1 to 1.000 by epoch 70, while validation F1 peaked at 0.9455 at epoch 2 and steadily degraded thereafter, falling to approximately 0.840 by epoch 30 before the process terminated due to an out-of-memory crash at epoch 70.

This pattern is attributable to the limited diversity of the training set relative to the capacity of the BEV CNN. With only 57 training clips, and with episodes 0–18 producing entirely empty BEV grids due to a coordinate-system error in early data collection (in which LiDAR points were recorded in world-scale rather than ego-centric coordinates, placing them outside the BEV

grid bounds), the effective training diversity for the BEV branch is limited to approximately 18 unique non-empty spatial configurations. The BEV CNN, with approximately 89,000 trainable parameters, is able to memorize this limited set of configurations within a small number of epochs, after which validation performance — measured on genuinely unseen episodes — degrades even as training performance continues to improve.

Despite this pattern, the epoch-2 checkpoint is the legitimate best model, for three reasons. First, the validation episodes ($\{3, 6, 45, 46\}$) share no road layout or NPC trajectory with the training set, so the validation F1 of 0.9455 reflects genuine generalization rather than memorization. Second, the reported test F1 of 0.9003 was independently verified by a separate evaluation script applied to the test episodes ($\{9, 10, 47, 49\}$), which were not used at any point during training or threshold calibration beyond their role as the final held-out set. Third, the relationship between the validation F1 (0.9455) and the test F1 (0.9003) is consistent with ordinary sampling variation given the smaller size of the test set and its different class balance, rather than indicating that the reported result is an artifact of overfitting. Early stopping based on validation performance ensured that this checkpoint was automatically saved before the more severe overfitting observed in later epochs occurred.

5.9. Generalization to Unseen Road Layouts

The episode-level train/validation/test split described in Chapter 3 ensures that the test F1 of 0.9003 reflects genuine generalization: the model has not encountered the road topology, lane geometry, or background scene of any test episode at any point during training. This is the central practical advantage of MetaDrive's procedural generation over fixed-map simulation platforms such as CARLA — each episode constitutes a structurally distinct environment, making the generalization claim substantially stronger than would be possible if multiple episodes shared a small number of fixed maps.

5.10. Time-to-Collision Safety Analysis

The F1, precision, and recall metrics reported in Sections 5.4–5.6 weight every frame within a labelled event window equally, regardless of how physically imminent the danger is at that specific frame. To assess Early Fusion V2's behavior from a safety perspective rather than a purely statistical one, a kinematic Time-to-Collision (TTC) measure is computed for every test-set frame. For each nearby vehicle, a closing speed is computed by projecting the relative world-frame velocity vector onto the ego-to-vehicle direction; where this closing speed exceeds 0.5 m/s (i.e. the vehicle is converging on the ego vehicle rather than diverging or moving in parallel), TTC is taken as distance divided by closing speed, as formalized in Equations 5.5–5.8 below. The minimum TTC across all nearby vehicles is retained per frame, and frames containing no converging vehicles are assigned a capped value of 30 seconds. A TTC below three seconds is treated as the safety-critical zone in this analysis, consistent with collision-warning thresholds commonly used in automotive safety literature [21], [22].

Let p_i and v_i denote the world-frame position and velocity of nearby vehicle i , and p_{ego} and v_{ego} the position and velocity of the ego vehicle. The distance to vehicle i is given by Equation 5.4:

$$d_i = |p_i - p_{ego}| \quad (5.4)$$

The closing speed is the negative projection of the relative velocity onto the unit line-of-sight vector toward vehicle i , as given in Equation 5.5; a positive value indicates the vehicle is converging on the ego vehicle:

$$v_{c,i} = -(v_i - v_{ego}) \cdot \frac{(p_i - p_{ego})}{d_i} \quad (5.5)$$

When $v_{c,i}$ exceeds the 0.5 m/s convergence threshold, the time-to-collision for vehicle i follows Equation 5.6:

$$TTC_i = \frac{d_i}{v_{c,i}} \quad (5.6)$$

Finally, the frame-level TTC retained for the analysis below is the minimum over all nearby vehicles, as given in Equation 5.7:

$$TTC_{frame} = \min_i TTC_i \quad (5.7)$$

Fig. 5.12 groups every test-set frame by prediction outcome and plots the resulting TTC distribution. The four groups contain 1,384 true positives, 128 false positives, 69 false negatives, and 116 true negatives; the true-positive and false-negative counts sum to 1,453 and the false-positive and true-negative counts sum to 244, exactly matching the rash-event and normal class supports reported in Table 5.1, confirming this analysis was performed on the identical held-out test set. All four groups span the full range from sub-one-second to the 30-second cap, reflecting that the binary event label (Chapter 3, Section 3.6) is applied across a multi-second window that includes both the brief closing-speed phase of a maneuver and the surrounding approach and recovery frames, during which no vehicle is necessarily on a converging trajectory. The median TTC is noticeably lower for false positives (approximately 3–4 seconds) than for false negatives (approximately 6 seconds), suggesting that the model's false alarms tend to occur in genuinely closer, more kinematically ambiguous situations, while its missed detections skew toward comparatively less acute moments within the labelled window.

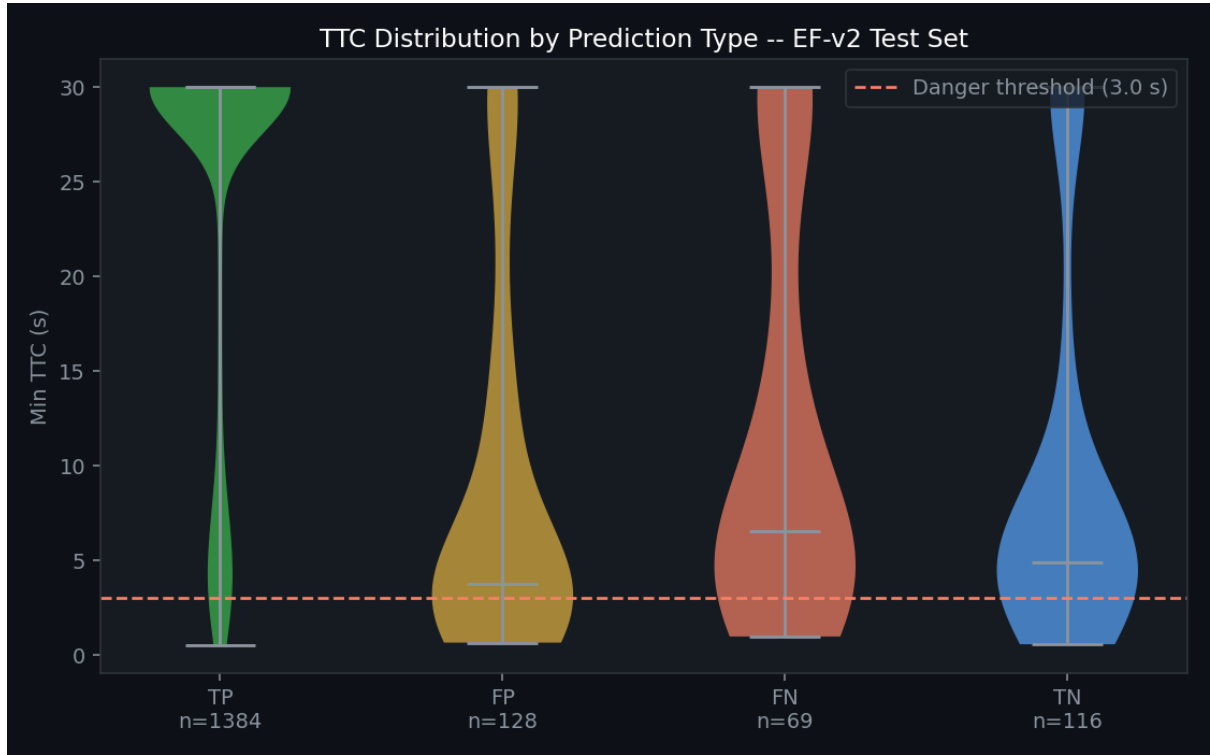


Fig 5.12. Violin plot of minimum TTC, grouped by prediction outcome (TP/FP/FN/TN) on the test set, with a dashed line marking the 3-second danger threshold.

Fig. 5.13 examines the trade-off between detection safety margin and recall as the decision threshold varies. Mean TTC at detection rises only modestly across the swept range — from approximately 24 seconds at a permissive threshold of 0.05 to approximately 28 seconds at a conservative threshold of 0.95 — while the true-positive count falls substantially, from over 1,400 to under 950. This asymmetry indicates that raising the decision threshold to obtain a marginally larger average safety margin comes at a disproportionate cost in missed detections, supporting the choice of an F1-optimised rather than a safety-margin-maximized operating threshold for this dissertation's reported results.

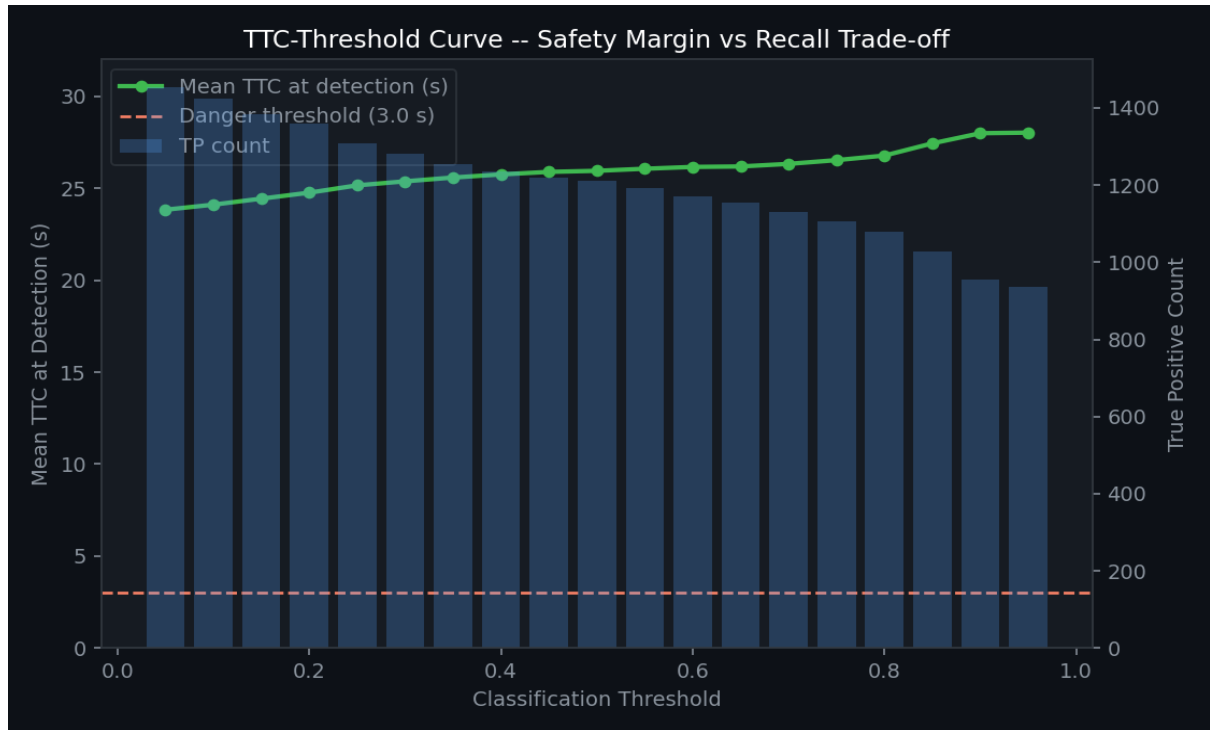


Fig 5.13 Mean TTC at true-positive detections plotted against classification threshold from 0.05 to 0.95, with true-positive count overlaid as a bar series.

Fig. 5.14 disaggregates detected and missed events by TTC. The majority of both groups fall in the highest TTC bin (no vehicle currently converging), consistent with the breadth of the labelled event window discussed above. A small minority of the 69 missed frames fall within the three-second safety-critical zone, visible as the short bars to the left of the marked threshold; this is the safety-relevant subset of the model's missed detections — cases where a rash event was both underway and kinematically imminent at the moment of the miss. This safety-critical subset represents the cases of greatest practical concern, as they correspond to genuine imminent danger that the model failed to detect.

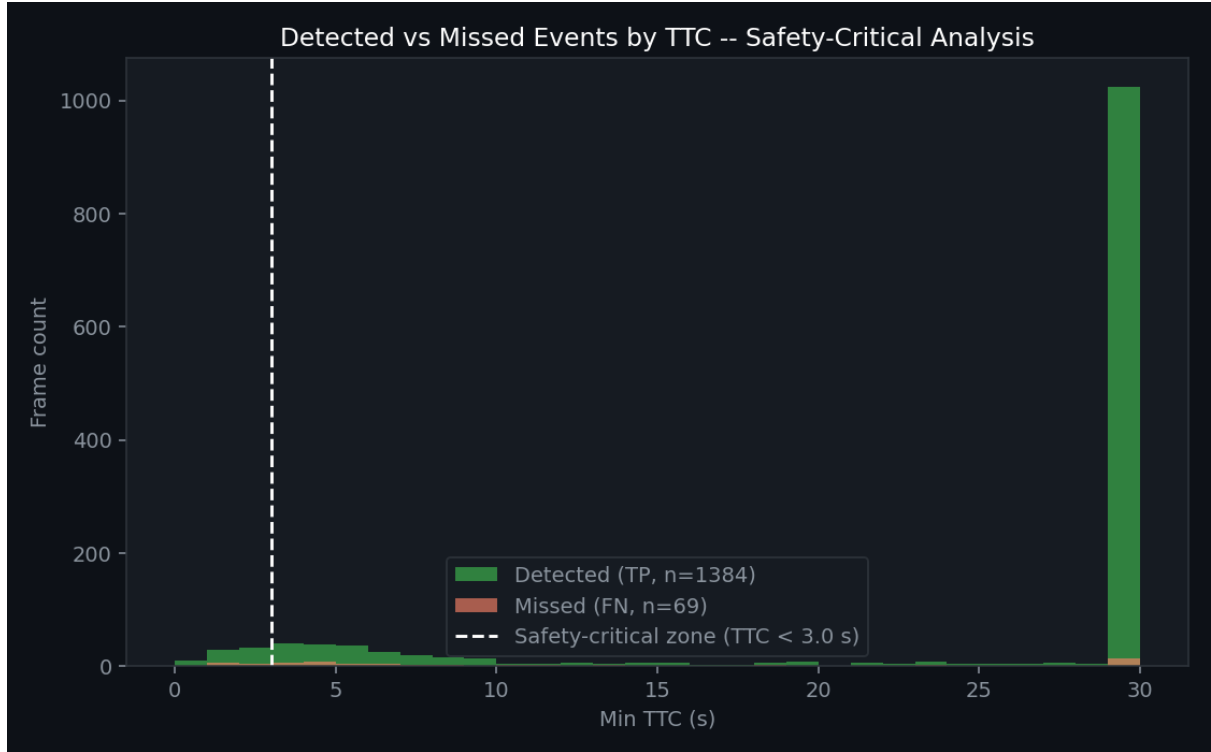


Figure 5.14. Histogram comparing detected (TP) and missed (FN) event frames by minimum TTC, with the 3-second safety-critical zone marked.

5.11. Computational Efficiency

The complete inference pipeline for Early Fusion V2 runs on CPU hardware without a dedicated GPU. The frozen MobileNetV3-Small visual backbone is lightweight (approximately 1.5 million parameters, operating on 224×224 inputs), and the trainable components — the BEV CNN ($\sim 89,000$ parameters) and the Bidirectional GRU — are compact by deliberate design choice. This computational profile was a primary design constraint rather than an incidental property: real AV onboard processors, such as the NVIDIA Drive, Qualcomm Snapdragon Ride, or Mobileye EyeQ platforms, are purpose-built systems-on-chip with limited computing budgets relative to a desktop GPU. Demonstrating that a system achieving 90% test F1 can be trained and run entirely on CPU establishes a more direct path

toward eventual deployment on this class of hardware than would a comparable system requiring substantial GPU acceleration.

5.12. Discussion

Taken together, the three models support several conclusions relevant to the design of third-party rash event prediction systems. Kinematic motion data alone is a remarkably strong predictor of rash behavior, suggesting that any deployed system should, at minimum, incorporate accurate relative position and velocity tracking of nearby vehicles. However, naively adding visual and statistically summarized LiDAR information does not improve on this baseline, and may degrade test-set performance — the failure of Early Fusion V1 demonstrates that multi-modal fusion is not unconditionally beneficial and depends critically on how each modality is represented. Preserving spatial structure in the LiDAR representation, through a CNN-processed BEV grid rather than scalar statistics, is necessary for multi-modal fusion to exceed the kinematic-only baseline. The resulting system, Early Fusion V2, achieves the strongest overall result and demonstrates that this benefit persists under a genuine episode-level generalization test. The TTC analysis in Section 5.10 further indicates that this benefit is not merely statistical: the model's missed detections skew toward less kinematically acute moments within the labelled event window, and the safety margin gained by adopting a more conservative decision threshold is small relative to the recall it sacrifices, reinforcing the appropriateness of the F1-optimised operating threshold used throughout this chapter.

5.13. Summary

This chapter presented the experimental results for all three models, an ablation analysis isolating the contribution of spatial LiDAR representation, an analysis of the overfitting pattern observed in Early Fusion V2 training and the basis for treating its epoch-2 checkpoint as valid,

an assessment of the system's generalization and computational efficiency, and a Time-to-Collision safety analysis contextualizing the model's detections and misses in physical terms. Chapter 6 concludes this dissertation and discusses directions for future research.

Chapter 6

Conclusion and Future Work

6.1. Summary of Contributions

This dissertation addressed third-party rash driving prediction — anticipating rash maneuvers in nearby vehicles using only an ego AV's own onboard sensor suite — a task framing distinct from the predominantly ego-centric aggressive driving detection literature. The contributions of this work are fourfold. First, a novel task framing was proposed and motivated: reactive threat anticipation, in contrast to ego-driver monitoring. Second, a purpose-built simulation dataset was constructed in MetaDrive, providing frame-precise binary event labels and four simultaneous sensor modalities (RGB, semantic segmentation, LiDAR BEV, kinematics) across 50 procedurally generated episodes — a combination of properties not available in any existing public dataset. Third, a four-modality temporal fusion architecture, Early Fusion V2, was designed and evaluated, achieving a test F1 of 0.9003 on road environments unseen during training. Fourth, this result was demonstrated to be achievable entirely on CPU hardware, without a dedicated GPU, supporting the practical deployability of the approach on constrained onboard AV computing platforms.

6.2. Limitations

Several limitations qualify the conclusions of this dissertation. The dataset, while purpose-built and novel in its label precision and modality combination, is modest in scale (50 episodes, 17,559 frames) relative to large-scale AV perception datasets, and this scale constraint is directly implicated in the overfitting behavior observed during Early Fusion V2 training (Chapter 5, Section 5.8). All data is collected in simulation; no experiments in this dissertation evaluate transfer to real-world sensor data, and the simulation-to-real domain gap — partially

but not fully addressed through the use of a frozen ImageNet-pretrained visual backbone — remains an open question. The binary rash/normal label does not distinguish between cut-in and emergency-braking event types, limiting the granularity of any downstream corrective response that might be designed around the model's output. Finally, the scope of this work is limited to vehicle-to-vehicle interactions; pedestrian and cyclist behavior, which present materially different motion and appearance characteristics, are not addressed.

6.3. Future Work

Several directions for future work follow directly from the limitations above. Sim-to-real transfer — validating whether features learned in MetaDrive generalize to real dashcam or AV sensor footage, potentially through domain adaptation techniques — represents the most direct extension of this work toward real-world deployment. Per-type classification, distinguishing cut-in events from emergency-braking events rather than collapsing both into a single binary label, would enable more precisely tailored corrective responses and is a natural extension given that the simulator already logs event type at collection time. Extending the prediction horizon — anticipating rash events one to two seconds before onset rather than at the moment of onset, as in the present work — would further increase the time available for a corrective AV response, though this would require careful handling of the inherent label uncertainty that comes with predicting further into the future. Extending the dataset to include pedestrian and cyclist actors would broaden the applicability of the approach to a wider range of AV safety scenarios. Finally, increasing dataset scale and, in particular, BEV spatial diversity — directly addressing the limited number of unique non-empty BEV configurations identified as the cause of the overfitting pattern in Chapter 5 — is likely to yield further performance improvements and a more robust training dynamic.

6.4. Concluding Remarks

This dissertation has shown that predicting rash driving behavior in nearby vehicles, using only an ego AV's own onboard multi-modal sensor suite, is achievable with strong performance (test $F1 = 0.9003$) and that this result depends critically on preserving the spatial structure of LiDAR information rather than reducing it to scalar statistics. The work establishes both a dataset and an architectural foundation for a research direction — outward-facing, third-party behavioral anticipation — that the existing literature has largely left unaddressed and identifies clear next steps toward closing the gap between this simulation-based result and real-world AV deployment.

REFERENCES

- [1] SAE International Recommended Practice, Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles, SAE Standard J3016_202104, Revised April 2021, Issued January 2014, https://doi.org/10.4271/J3016_202104.
- [2] Autonomous vehicle market size worldwide 2021-2030, Statista Research Department (2025). https://www.statista.com/statistics/1224515/av-market-size-worldwide-forecast/?srsltid=AfmBOoq_Q4RSeh7xSRqJeeG3x_vvhIOh51nLITpWk7-zZEIajJ8gQ9zn
- [3] *Smartphone-based driving behaviour classification using accelerometer and gyroscope sensors.*
- [4] *Smartphone-based sensing platform for modeling aggressive driving behaviors using accelerometer, gyroscope, magnetometer, and GPS fusion.*
<https://www.researchgate.net/publication/266655676>
- [5] *Decision-tree and HMM-based abnormal driving behavior classification from inertial sensor streams.* <https://ieeexplore.ieee.org/document/9251913/>
- [6] *"A Novel IMU-Based Aggressiveness Index for Driver Behavior Assessment Using Wearable Sensing,"* Machines, 2025. doi: 10.3390/machines14060582
- [7] K. Huang, B. Shi, X. Li, X. Li, S. Huang, and Y. Li, "Multi-modal Sensor Fusion for Auto Driving Perception: A Survey," arXiv:2202.02703, 2022. <https://arxiv.org/abs/2202.02703>
- [8] Z. Liu, H. Tang, A. Amini, X. Yang, H. Mao, D. Rus, and S. Han, *"BEVFusion: Multi-Task Multi-Sensor Fusion with Unified Bird's-Eye View Representation,"* Proc. ICRA, 2023. arXiv:2205.13542. <https://arxiv.org/abs/2205.13542>
- [9] K. Chitta, A. Prakash, B. Jaeger, Z. Yu, K. Renz, and A. Geiger, *"TransFuser: Imitation with Transformer-Based Sensor Fusion for Autonomous Driving,"* IEEE Trans. Pattern Anal. Mach. Intell., 2022. arXiv:2205.15997. <https://arxiv.org/abs/2205.15997>

- [10] S. Vora, A. H. Lang, B. Helou, and O. Beijbom, "*PointPainting: Sequential Fusion for 3D Object Detection*," Proc. CVPR, 2020. arXiv:1911.10150. <https://arxiv.org/abs/1911.10150>
- [11] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom, "*nuScenes: A Multimodal Dataset for Autonomous Driving*," in Proc. CVPR, 2020. arXiv:1903.11027. <https://arxiv.org/abs/1903.11027>
- [12] M. Simon, K. Amende, A. Kraus, J. Honer, T. S  mann, H. Kaulbersch, S. Milz, and H. M. Gross, "*Complexer-YOLO: Real-Time 3D Object Detection and Tracking on Semantic Point Clouds*," arXiv:1904.07537, 2019. <https://arxiv.org/abs/1904.07537>
- [13] A. Alahi, K. Goel, V. Ramanathan, A. Robicquet, L. Fei-Fei, and S. Savarese, "*Social LSTM: Human Trajectory Prediction in Crowded Spaces*," Proc. CVPR, 2016. https://cvgl.stanford.edu/papers/CVPR16_Social_LSTM.pdf
- [14] N. Deo and M. M. Trivedi, "*Convolutional Social Pooling for Vehicle Trajectory Prediction*," Proc. CVPR Workshops, 2018. arXiv:1805.06771. <https://arxiv.org/abs/1805.06771>
- [15] "*Driving Intention Recognition of Surrounding Vehicles Based on a Time-Sequenced Weights Hidden Markov Model for Autonomous Driving*," Sensors, vol. 23, no. 21, 2023. <https://www.mdpi.com/1424-8220/23/21/8761>
- [16] Q. Liu, H. Huang, S. Zhao, L. Shi, S. Ahn, and X. Li, "*RiskNet: Interaction-Aware Risk Forecasting for Autonomous Driving in Long-Tail Scenarios*," 2025. arXiv:2504.15541. <https://arxiv.org/abs/2504.15541>
- [17] W. Ben, B. Li, and G. Yin, "*Recognition of Surrounding Vehicle Anomalous Intent in the Vehicle-Road-Cloud Environment for Autonomous Driving*," 2025. doi: 10.1016/j.cjme.2025.100169. <https://www.sciencedirect.com/science/article/pii/S1000934525001737>

- [18] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "*CARLA: An Open Urban Driving Simulator*," Proc. CoRL, 2017.
- [19] Q. Li, Z. Peng, L. Feng, Q. Zhang, Z. Xue, and B. Zhou, "*MetaDrive: Composing Diverse Driving Scenarios for Generalizable Reinforcement Learning*," IEEE Trans. Pattern Anal. Mach. Intell., 2022. arXiv:2109.12674. <https://arxiv.org/abs/2109.12674>
- [20] F. Yang, B. Yu, Y. Zhou, X. Luo, Z. Tu, and C. Liu, "*REACT: Edge-Based Multimodal Sensor Data Fusion with Vision Language Models for Real-Time Autonomous Vehicle Accident Avoidance*," arXiv:2508.01057, 2025. <https://arxiv.org/abs/2508.01057>
- [21] J. C. Hayward, "*Near-miss determination through use of a scale of danger*," Highway Research Record, no. 384, pp. 24–34, 1972.
- [22] M. M. Minderhoud and P. H. L. Bovy, "*Extended time-to-collision measures for road traffic safety assessment*," Accident Analysis & Prevention, vol. 33, no. 1, pp. 89–97, 2001.